

C PROGRAMMING / COMPLETE EXAMPLE HANDBOOK

從完整題目開始， 把解題思路落實成可測試的程式。

整合選擇敘述、迴圈與基本型別。每題先定義輸入與輸出，再逐步拆解流程，提供完整 C 程式及一般、邊界、錯誤路徑的具體測試。

16	20	37	262
正式實際案例	明確標示的語法補充	可獨立編譯的完整程式	完整輸入 / 輸出測試資料

● 本次 **Linux** 實測 GCC 269/269 Clang 269/269 GCC + UBSan 269/269 判準與教學警告 →

教材沿用與新增：C01–C16 是整合版原有正式案例；S01–S20 是將語法片段 / 除錯練習補成可執行題目。原有17份程式完整保留。平台觀察值另行標示，不當成所有電腦的固定答案。

跳至主要內容

C01

正式案例

選擇敘述

C11

成績等第判定

if

else if

區間

char

對應整合版：第 10–12 頁 · 1 份程式 · 14 組測試

來源對照：Ch5 pp.15–29：if / else if 與區間判斷

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

為課程成績系統撰寫單筆等第判定程式。讀入一個整數成績，先驗證範圍，再依規則只輸出一個等第。

輸入規格

一個整數 score；合法範圍 0 ~ 100。

輸出規格

合法輸入輸出 A、B、C、D 或 F；超出範圍輸出 ERR_RANGE；無法讀入整數輸出 ERR_INPUT。

規則與限制

- 90 ~ 100：A；80 ~ 89：B；70 ~ 79：C。
- 60 ~ 69：D；0 ~ 59：F。
- 同一筆成績只能命中一個等第；上下界皆包含。

輸入 / 輸出範例

STDIN · 範例輸入

85

3 bytes · 最後有 LF 換行

STDOUT · 範例輸出

B

2 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變

！ 跳至主要內容
讀取 score

STEP 02

排除範圍錯誤

STEP 03

由高到低測門檻

STEP 04

印出唯一等第

- 1
- 先把不合法成績排除，後面的分支才共享 $0 \leq \text{score} \leq 100$ 的前提。
- 2
- 門檻由高到低測試；85 在 ≥ 90 失敗後，於 ≥ 80 命中。
- 3
- else if 只執行第一個成立分支，避免多個獨立 if 覆寫結果。

重要變數與狀態

名稱	型別 / 形式	用途
score	int	待判定成績
grade	char	最後選定的一個等第

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

檢查	代入 85	行為
輸入範圍	$0 \leq 85 \leq 100$	通過
$\text{score} \geq 90$	假	繼續
$\text{score} \geq 80$	真	grade = B
其餘 else if	不再測試	輸出 B

為什麼這個解法成立？

有效輸入會落在互斥且完整的五個區間中。依序使用 else if，第一個成立分支決定等第，後面的門檻不再執行。

時間 $O(1)$ ；額外空間 $O(1)$ 。

03

完整程式碼

每一份都可獨立編譯

c01_grade.c

完整程式 · 18 行 · 長行可橫向捲動

```
1>
跳至主要內容
#include <stdio.h>
char grade;
if (scanf("%d", &score) != 1) {
    puts("ERR_INPUT"); return 1;
}
```

```
    }
    if (score < 0 || score > 100) {
        puts("ERR_RANGE"); return 1;
    }
    if (score >= 90) grade = 'A';
    else if (score >= 80) grade = 'B';
    else if (score >= 70) grade = 'C';
    else if (score >= 60) grade = 'D';
    else grade = 'F';
    printf("%c\n", grade);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c01_grade.c -o c01_grade
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

14 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

C01-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
85	B
3 bytes · 最後有 LF 換行	2 bytes · 包含程式實際輸出的換行與空白

C01-T02 合法最小值0

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0 跳至主要內容	F
2 bytes · 最後有 LF 換行	2 bytes · 包含程式實際輸出的換行與空白

–

C01-T03 F/D門檻前一分

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

59

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

F

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T04 剛好60分、由F進入D

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

60

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

D

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T05 D/C門檻前一分

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

69

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

D

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T06 剛好70分、由D進入C

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

70

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

C

2 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

–

C01-T07 C/B門檻前一分

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

79

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

C

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T08 剛好80分、由C進入B

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

80

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

B

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T09 B/A門檻前一分

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

89

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

B

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T10 剛好90分、由B進入A

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

90

跳至主要內容

↵ 換行

STDOUT · 完整預期輸出

A

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T11 合法最大值100

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

100

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

A

2 bytes · 包含程式實際輸出的換行與空白

–

C01-T12 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

–

C01-T13 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

101

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

–

C01-T14 驗證 ERR_INPUT

exit 1 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

跳至主要內容

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- 用 59/60、89/90 檢查邊界，而不只測 85。
- grade 用 char 儲存單一字元；字元常數用單引號。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

分段手續費計算

if

費率

double

邊界值

對應整合版：第 13–16 頁 · 1 份程式 · 19 組測試

來源對照：Ch5 pp.30–32：broker.c

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

依原教材的假設費率，計算一筆交易的手續費。先找交易金額所屬區間，套用該區間公式，最後套用最低收費。這是教學假設，並非現行券商費率。

輸入規格

一個有限實數 $value$ ， $0 < value \leq 1,000,000,000$ 。

輸出規格

輸出 Commission: 金額，顯示到小數第 2 位；非法範圍輸出 ERR_RANGE；讀取失敗輸出 ERR_INPUT。

- 規則與限制
- [0,2500)： $30 + 0.017v$ ；[2500,6250)： $56 + 0.0066v$ 。
 - [6250,20000)： $76 + 0.0034v$ ；[20000,50000)： $100 + 0.0022v$ 。
 - [50000,500000)： $155 + 0.0011v$ ；[500000,∞)： $255 + 0.0009v$ 。
 - 完成分段計算後，低於 39 一律改收 39；不採累進分段加總。

輸入 / 輸出範例

STDIN · 範例輸入

30000

6 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Commission: 166.00

19 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

核心思考與步驟

- 1
- 把區間視為左閉右開，直接沿用原程式的 < 上界規則。
- 2
- 30000 命中 [20000,50000)，得到 $100 + 0.0022 \times 30000 = 166$ 。
- 3
- 最低收費是第二層規則，不是另一個金額區間。

重要變數與狀態

名稱	型別 / 形式	用途
value	double	交易金額
fee	double	套用公式後的手續費

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

步驟	計算 / 判斷	結果
選區間	$20000 \leq 30000 < 50000$	使用 $100 + 0.0022v$
代公式	$100 + 0.0022 \times 30000$	166
最低收費	$166 < 39$ 為假	保持 166
格式化	<code>%.2f</code>	Commission: 166.00

為什麼這個解法成立？

所有通過驗證的金額只使用一個區間公式。最低收費在區間判斷完成後獨立套用，所以每筆正常輸出的金額不小於 39。表格描述費率區間；合法輸入仍受 $0 < \text{value} \leq 10^9$ 限制。

時間 $O(1)$ ；額外空間 $O(1)$ 。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

c02_commission.c

完整程式 · 25 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    double value, fee;
    if (scanf("%lf", &value) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (!(value > 0.0 && value <= 1.0e9)) {
        puts("ERR_RANGE"); return 1;
    }
    if (value < 2500.0)
        fee = 30.0 + 0.017 * value;
    else if (value < 6250.0)
        fee = 56.0 + 0.0066 * value;
    else if (value < 20000.0)
        fee = 76.0 + 0.0034 * value;
    else if (value < 50000.0)
        fee = 100.0 + 0.0022 * value;
    else if (value < 500000.0)
        fee = 155.0 + 0.0011 * value;
    else
        fee = 255.0 + 0.0009 * value;
    if (fee < 39.0) fee = 39.0;
    printf("Commission: %.2f\n", fee);
    return 0;
}
```

單題編譯（在包含本檔的資料夾執行）：

```
gcc -std=c11 -Wall -Wextra -Wpedantic c02_commission.c -o c02_commission
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

19 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C02-T01 題目示範輸入 / 輸出

exit 0 ✓

跳至主要內容 [↗ · 6 input bytes · 完整比對](#)

STDIN · 完整輸入

STDOUT · 完整預期輸出

30000

6 bytes · 最後有 LF 換行

Commission: 166.00

19 bytes · 包含程式實際輸出的換行與空白

C02-T02 計算值低於39，套用最低收費

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

100

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 39.00

18 bytes · 包含程式實際輸出的換行與空白

C02-T03 2500屬第二區間的下界

exit 0 ✓

原資源包 · 5 input bytes · 完整比對

STDIN · 完整輸入

2500

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 72.50

18 bytes · 包含程式實際輸出的換行與空白

C02-T04 6250屬第三區間的下界

exit 0 ✓

原資源包 · 5 input bytes · 完整比對

STDIN · 完整輸入

6250

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 97.25

18 bytes · 包含程式實際輸出的換行與空白

C02-T05 20000屬第四區間的下界

exit 0 ✓

原資源包 · 6 input bytes · 完整比對

跳至主要內容

20000

STDOUT · 完整預期輸出

Commission: 144.00

6 bytes · 最後有 LF 換行

19 bytes · 包含程式實際輸出的換行與空白

- C02-T06

50000屬第五區間的下界

原資源包 · 6 input bytes · 完整比對

exit 0 ✓

STDIN · 完整輸入

50000

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 210.00

19 bytes · 包含程式實際輸出的換行與空白

- C02-T07

500000屬最後區間的下界

原資源包 · 7 input bytes · 完整比對

exit 0 ✓

STDIN · 完整輸入

500000

7 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 705.00

19 bytes · 包含程式實際輸出的換行與空白

- C02-T08

驗證 ERR_RANGE

原資源包 · 2 input bytes · 完整比對

exit 1 ✓

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

- C02-T09

驗證 ERR_RANGE

原資源包 · 4 input bytes · 完整比對

exit 1 ✓

跳至主要內容

STDIN · 完整輸入
-10
4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

- C02-T10 驗證 ERR_RANGE

exit 1 ✓
- 原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入
nan
4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

- C02-T11 非數字 token：scanf 讀取失敗

exit 1 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入
x
2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

- C02-T12 最大合法交易金額

exit 0 ✓
- 本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入
1000000000
11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Commission: 900255.00
22 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容: 金額超出上限

exit 1 ✓
- L 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入

1000000001

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

- C02-T14

2500 門檻前一個整數

exit 0 ✓
- 本 HTML 新增測試 · 5 input bytes · 完整比對

STDIN · 完整輸入

2499

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 72.48

18 bytes · 包含程式實際輸出的換行與空白

- C02-T15

6250 門檻前一個整數

exit 0 ✓
- 本 HTML 新增測試 · 5 input bytes · 完整比對

STDIN · 完整輸入

6249

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 97.24

18 bytes · 包含程式實際輸出的換行與空白

- C02-T16

20000 門檻前一個整數

exit 0 ✓
- 本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入

19999

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Commission: 144.00

19 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容

門檻前一個整數

exit 0 ✓
- L 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入
49999
6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Commission: 210.00
19 bytes · 包含程式實際輸出的換行與空白

–

C02-T18 500000 門檻前一個整數

exit 0 ✓

本 HTML 新增測試 · 7 input bytes · 完整比對

STDIN · 完整輸入
499999
7 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Commission: 705.00
19 bytes · 包含程式實際輸出的換行與空白

–

C02-T19 正無窮值不屬於合法有限範圍

exit 1 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入
inf
4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 原表格端點較含糊，本版以原始程式的比較式界定邊界。
- 使用 double 與 %.2f 只是本題顯示方式，不保證十進位金額完全精確。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制與測試規則。

跳至主要內容

四則計算器

switch

除以零

cast

運算子

對應整合版：第 20–22 頁 · 1 份程式 · 12 組測試

來源對照：Ch5 pp.75–76：計算器；pp.58–68：switch

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

讀入兩個整數與中間的運算符號，用 switch 完成 +、-、*、/ 四則運算。除法必須得到小數結果，並處理除以零與未知符號。

輸入規格

格式為 `a op b`；`a`、`b` 為 -10000 ~ 10000 的整數，`op` 為單一字元。

輸出規格

正常輸出 `Result: 數值 ( 小數 2 位 )`；其他輸出 `ERR_INPUT`、`ERR_RANGE`、`ERR_ZERO` 或 `ERR_OP`。

規則與限制

- 只接受 +、-、*、/。
- 除法先檢查 `b` 是否為 0；其他三種運算允許 `b=0`。
- 先將 `a` 轉成 `double`，使計算不依賴 `int` 乘積能否容納結果。

輸入 / 輸出範例

STDIN · 範例輸入

7 / 2

6 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Result: 3.50

13 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變

！ 跳至主要內容

讀取 `a op b`

STEP 02

共同範圍驗證

STEP 03

switch 選運算

STEP 04

成功輸出或回報錯誤

核心思考與步驟

- 1
- 輸入成功後先驗證共同範圍，再依 `op` 分派。
- 2
- 除以零隻屬於 / 分支；不要讓合法的 `7+0` 被拒絕。
- 3
- 每個成功 `case` 都設定 `result`；錯誤路徑 `return`，避免讀未初始化值。

重要變數與狀態

名稱	型別 / 形式	用途
<code>a / b</code>	<code>int</code>	左右運算元
<code>op</code>	<code>char</code>	單一運算符號
<code>result</code>	<code>double</code>	成功運算的結果

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

步驟	7 / 2	說明
選擇分支	<code>case '/'</code>	只做除法
檢查分母	<code>2 != 0</code>	可進行
先轉型	<code>(double)7 / 2</code>	以浮點除法計算
結果	3.5	輸出 <code>Result: 3.50</code>

為什麼這個解法成立？

每個正常 `case` 都會設定 `result`；非法符號與零分母會先 `return`，不會使用尚未初始化的結果。共同範圍檢查先於運算符號檢查。

時間 $O(1)$ ；額外空間 $O(1)$ 。

03

完整程式碼

每一份都可獨立編譯

跳至主要內容

可橫向捲動

```
#include <stdio.h>
int main(void) {
```

```
int a, b;
char op;
double result;
if (scanf("%d %c %d", &a, &op, &b) != 3) {
    puts("ERR_INPUT"); return 1;
}
if (a < -10000 || a > 10000 ||
    b < -10000 || b > 10000) {
    puts("ERR_RANGE"); return 1;
}
switch (op) {
case '+': result = (double)a + b; break;
case '-': result = (double)a - b; break;
case '*': result = (double)a * b; break;
case '/':
    if (b == 0) { puts("ERR_ZERO"); return 1; }
    result = (double)a / b; break;
default: puts("ERR_OP"); return 1;
}
printf("Result: %.2f\n", result);
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c03_calculator.c -o c03_calculator
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

12 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

C03-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 6 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
7 / 2	Result: 3.50
6 bytes · 最後有 LF 換行	13 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

—

C03-T02 加法分支

exit 0 ✓

原資源包 · 6 input bytes · 完整比對

STDIN · 完整輸入

4 + 5

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Result: 9.00

13 bytes · 包含程式實際輸出的換行與空白

- C03-T03 減法結果為負數

exit 0 ✓
- 原資源包 · 6 input bytes · 完整比對

STDIN · 完整輸入

4 - 5

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Result: -1.00

14 bytes · 包含程式實際輸出的換行與空白

- C03-T04 最大運算元相乘 · 避免先做int乘法

exit 0 ✓
- 原資源包 · 14 input bytes · 完整比對

STDIN · 完整輸入

10000 * 10000

14 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Result: 100000000.00

21 bytes · 包含程式實際輸出的換行與空白

- C03-T05 驗證 ERR_ZERO

exit 1 ✓
- 原資源包 · 6 input bytes · 完整比對

STDIN · 完整輸入

7 / 0

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_ZERO

9 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容 RR_OP

exit 1 ✓
- 🔗 · 6 input bytes · 完整比對

STDIN · 完整輸入

7 % 2

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_OP

7 bytes · 包含程式實際輸出的換行與空白

- C03-T07 加法允許第二運算元為0

exit 0 ✓
- 原資源包 · 6 input bytes · 完整比對

STDIN · 完整輸入

7 + 0

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Result: 7.00

13 bytes · 包含程式實際輸出的換行與空白

- C03-T08 驗證 ERR_INPUT

exit 1 ✓
- 原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

- C03-T09 第一個運算元超出範圍

exit 1 ✓
- 本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入

10001 + 1

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容 | 運算元超出範圍

exit 1 ✓
- L 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入

1 + -10001

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

–

C03-T11 負數浮點除法

exit 0 ✓

本 HTML 新增測試 · 7 input bytes · 完整比對

STDIN · 完整輸入

-7 / 2

7 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Result: -3.50

14 bytes · 包含程式實際輸出的換行與空白

–

C03-T12 缺少第二個運算元

exit 1 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

1 +

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 原例以 if 串接；本版改用 switch，新增錯誤路徑。
- (double)a / b 是「先轉型再運算」，第 7 章會再次剖析。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制與測試規則。

跳至主要內容

英文日期格式化

switch

日期

閏年

序數

對應整合版：第 23–26 頁 · 1 份程式 · 19 組測試

來源對照：Ch5 pp.69–71：date.c

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01 完整題目

先確認規格，再寫程式

把月/日/年的數字日期轉成英文日期句子。輸出前須確認日期存在，包括大小月及西元閏年；日期的英文序數字尾也必須正確。

輸入規格

格式 mm/dd/yyyy；year 範圍 1900 ~ 9999。月份與日期為整數。

輸出規格

合法時輸出 Dated this 日序數 day of 英文月份, 年.；格式錯誤輸出 ERR_INPUT；不存在的日期輸出 ERR_DATE。

規則與限制

- 閏年：能被 400 整除，或能被 4 整除且不能被 100 整除。
- 1、21、31 用 st；2、22 用 nd；3、23 用 rd；其餘用 th。
- 用完整四位數年份；1900 非閏年，2000 為閏年。

輸入 / 輸出範例

STDIN · 範例輸入

7/19/2026

10 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Dated this 19th day of July, 2026.

35 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

0 跳至主要內容 }

理解狀態如何改變

STEP 01

STEP 02

STEP 03

STEP 04

核心思考與步驟

- 1
- 日期驗證和文字輸出分開，避免先印出半句才發現無效日期。
- 2
- 先找該月天數；二月再套閏年條件。
- 3
- 合法日數僅 1~31，直接列 st/nd/rd 的 case，可自然排除 11、12、13。

重要變數與狀態

名稱	型別 / 形式	用途
m / d / y	int	月份、日、年
days	int	該月允許的最大日數

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

步驟	7/19/2026	結果
年月範圍	1900≤2026≤9999；m=7	通過
本月天數	七月不是 30 天月，也不是二月	days=31
日數範圍	1≤19≤31	通過
文字輸出	19 對應 th；7 對應 July	Dated this 19th day of July, 2026.

為什麼這個解法成立？

先驗證年月、再求正確天數、再驗證日，避免輸出不存在的日期。因合法日僅 1~31，列舉 1/21/31 等 case 能直接處理序數例外。

時間 O(1)；額外空間 O(1)。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

c04_date.c

完整程式 · 45 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int m, d, y, days = 31;
    if (scanf("%d /%d /%d", &m, &d, &y) != 3) {
        puts("ERR_INPUT"); return 1;
    }
    if (y < 1900 || y > 9999 || m < 1 || m > 12) {
        puts("ERR_DATE"); return 1;
    }
    switch (m) {
        case 4: case 6: case 9: case 11:
            days = 30; break;
        case 2:
            days = (y % 400 == 0 ||
                (y % 4 == 0 && y % 100 != 0)) ? 29 : 28;
            break;
    }
    if (d < 1 || d > days) {
        puts("ERR_DATE"); return 1;
    }
    printf("Dated this %d", d);
    switch (d) {
        case 1: case 21: case 31: printf("st"); break;
        case 2: case 22: printf("nd"); break;
        case 3: case 23: printf("rd"); break;
        default: printf("th"); break;
    }
    printf(" day of ");
    switch (m) {
        case 1: printf("January"); break;
        case 2: printf("February"); break;
        case 3: printf("March"); break;
        case 4: printf("April"); break;
        case 5: printf("May"); break;
        case 6: printf("June"); break;
        case 7: printf("July"); break;
        case 8: printf("August"); break;
        case 9: printf("September"); break;
        case 10: printf("October"); break;
        case 11: printf("November"); break;
        case 12: printf("December"); break;
    }
    printf(", %d.\n", y);
    return 0;
}
```

[跳至主要內容](#)

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c04_date.c -o c04_date
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

19 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

- C04-T01

題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

7/19/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 19th day of July, 2026.

35 bytes · 包含程式實際輸出的換行與空白

- C04-T02

2000可被400整除，是世紀閏年

exit 0 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

2/29/2000

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 29th day of February, 2000.

39 bytes · 包含程式實際輸出的換行與空白

- C04-T03

驗證 ERR_DATE

exit 1 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

2/29/1900

跳至主要內容 · F 換行

STDOUT · 完整預期輸出

ERR_DATE

9 bytes · 包含程式實際輸出的換行與空白

- C04-T04

驗證 ERR_DATE

exit 1 ✓

STDIN · 完整輸入

4/31/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_DATE

9 bytes · 包含程式實際輸出的換行與空白

—

C04-T05

11使用th · 不是st

exit 0 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

1/11/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 11th day of January, 2026.

38 bytes · 包含程式實際輸出的換行與空白

—

C04-T06

21使用st

exit 0 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

1/21/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 21st day of January, 2026.

38 bytes · 包含程式實際輸出的換行與空白

—

C04-T07

22使用nd

exit 0 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

1/22/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 22nd day of January, 2026.

38 bytes · 包含程式實際輸出的換行與空白

—

跳至主要內容

—

C04-T08

23使用rd

exit 0 ✓

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入
1/23/2026
10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Dated this 23rd day of January, 2026.
38 bytes · 包含程式實際輸出的換行與空白

- C04-T09 驗證 ERR_DATE

exit 1 ✓
- 原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入
13/1/2026
10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_DATE
9 bytes · 包含程式實際輸出的換行與空白

- C04-T10 驗證 ERR_DATE

exit 1 ✓
- 原資源包 · 9 input bytes · 完整比對

STDIN · 完整輸入
0/1/2026
9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_DATE
9 bytes · 包含程式實際輸出的換行與空白

- C04-T11 驗證 ERR_INPUT

exit 1 ✓
- 原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入
7-19-2026
10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容

exit 1 ✓
- ! token : scanf 讀取失敗
L 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

- C04-T13 一般閏年

exit 0 ✓
- 本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入

2/29/2024

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 29th day of February, 2024.

39 bytes · 包含程式實際輸出的換行與空白

- C04-T14 非閏年的二月 29 日

exit 1 ✓
- 本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入

2/29/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_DATE

9 bytes · 包含程式實際輸出的換行與空白

- C04-T15 12th 例外

exit 0 ✓
- 本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入

1/12/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 12th day of January, 2026.

38 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容

exit 0 ✓
- 12th 例外
L 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入

1/13/2026

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 13th day of January, 2026.

38 bytes · 包含程式實際輸出的換行與空白

– C04-T17 最大合法年份與月末

exit 0 ✓

本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入

12/31/9999

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Dated this 31st day of December, 9999.

39 bytes · 包含程式實際輸出的換行與空白

– C04-T18 年份低於下限

exit 1 ✓

本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入

1/1/1899

9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_DATE

9 bytes · 包含程式實際輸出的換行與空白

– C04-T19 不存在的第 0 日

exit 1 ✓

本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入

1/0/2026

9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_DATE

9 bytes · 包含程式實際輸出的換行與空白

錯誤 跳至主要內容 也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- 原例只示範格式化；本版新增日期驗證並改用四位年份。
- 12、13 分別是 12th、13th，不可只看個位數。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

找出足夠大的 2 次方容量

while

2次方

不變數

對應整合版：第 30–32 頁 · 1 份程式 · 9 組測試

來源對照：Ch6 p.8：最小的 2 次方

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

某緩衝區只能選擇 1、2、4、8、16...等容量。輸入需求 n ，找出不小於 n 的最小 2 次方容量 p 。

輸入規格

一個整數 n ， $1 \sim 1,000,000,000$ 。

輸出規格

輸出 Capacity: p ；讀取或範圍錯誤分別輸出 ERR_INPUT / ERR_RANGE。

規則與限制

- 初始容量為 1；不足時每次乘以 2。
- 需求剛好是 2 次方時，不再擴大。
- n 的上限使結果最大為 1,073,741,824；以 long long 計算。

輸入 / 輸出範例

STDIN · 範例輸入

10

3 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Capacity: 16

13 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



- 1 狀態只需 n 與 p ；每回合都保持 p 是 2 次方。
- 2 `while` 在執行本體之前判斷， $n=1$ 時不必倍增。
- 3 停止時 $p \geq n$ ；若 $p > 1$ ，前一個容量 $p/2 < n$ ，因此答案最小。

重要變數與狀態

名稱	型別 / 形式	用途
n	long long	容量需求
p	long long	目前候選容量，初值 1

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

檢查前 p	$p < 10$?	下一個狀態
1	是	$p=2$
2	是	$p=4$
4	是	$p=8$
8	是	$p=16$
16	否	停止；Capacity: 16

為什麼這個解法成立？

p 從 1 出發且只倍增，永遠是 2 的次方。停止時 $p \geq n$ ；若 $p > 1$ ，上一次 $p/2 < n$ ，所以不可能存在更小且足夠的 2 次方容量。

時間 $O(\log n)$ ；額外空間 $O(1)$ 。

03 完整程式碼

每一份都可獨立編譯

c05_power2.c

5 跳至主要內容

```
#include <stdio.h>
int main(void) {
    long long n, p = 1;
```

```
    if (scanf("%lld", &n) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (n < 1 || n > 1000000000LL) {
        puts("ERR_RANGE"); return 1;
    }
    while (p < n) {
        p *= 2;
    }
    printf("Capacity: %lld\n", p);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c05_power2.c -o c05_power2
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

9 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C05-T01

題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
10	Capacity: 16
3 bytes · 最後有 LF 換行	13 bytes · 包含程式實際輸出的換行與空白

— C05-T02

n=1時，while本體不執行

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1 跳至主要內容	Capacity: 1
2 bytes · 最後有 LF 換行	12 bytes · 包含程式實際輸出的換行與空白

–

C05-T03 需求已是2次方，不再倍增

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
16	Capacity: 16
3 bytes · 最後有 LF 換行	13 bytes · 包含程式實際輸出的換行與空白

–

C05-T04 剛超過16，容量須升至32

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
17	Capacity: 32
3 bytes · 最後有 LF 換行	13 bytes · 包含程式實際輸出的換行與空白

–

C05-T05 最大需求值與停止條件

exit 0 ✓

原資源包 · 11 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1000000000	Capacity: 1073741824
11 bytes · 最後有 LF 換行	21 bytes · 包含程式實際輸出的換行與空白

–

C05-T06 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0	ERR_RANGE
跳至主要內容 ↗ 換行	10 bytes · 包含程式實際輸出的換行與空白

–

C05-T07 非數字 token : scanf 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
x	ERR_INPUT
2 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

–

C05-T08 剛好達到第一次倍增值

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
2	Capacity: 2
2 bytes · 最後有 LF 換行	12 bytes · 包含程式實際輸出的換行與空白

–

C05-T09 需求超出上限

exit 1 ✓

本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1000000001	ERR_RANGE
11 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 本體改成 `p+=2` 會破壞「2 次方」不變數。
- 刪掉更新 `p*=2`，`n>1` 時將無法停止。

跳至主要內容

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

平方表：while 與 for 雙版本

while

for

平方

迴圈端點

對應整合版：第 33–36 頁 · 2 份程式 · 7 組測試

來源對照：Ch6 pp.13–14、43–46：square.c / square2.c / square3.c

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01 完整題目

先確認規格，再寫程式

讀入表格長度 n ，依序列印 $1 \sim n$ 的整數與平方。先完成 while 版本，再改寫為 for 版本，兩者輸出必須完全相同。

輸入規格

一個整數 n ， $1 \sim 20$ 。

輸出規格

每列輸出 i 與 i 的平方，中間一個空白，共 n 列；錯誤輸出 ERR_INPUT / ERR_RANGE。

規則與限制

- 包含 $i=1$ 與 $i=n$ ；不能多印第 0 列或第 $n+1$ 列。
- 平方使用 `1LL*i*i`，把乘法運算型別先提升。
- 兩個程式各有自己的 main，可分別編譯執行。

輸入 / 輸出範例

STDIN · 範例輸入

5

2 bytes · 最後有 LF 換行

STDOUT · 範例輸出

1 1
2 4
3 9
4 16
5 25

22 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

跳至主要內容

理解狀態如何改變

STEP 01

STEP 02

STEP 03

STEP 04



核心思考與步驟

- 1 初始化 $i=1$; 判斷 $i \leq n$; 印出 i 和平方 ; 最後 $i++$ 。
- 2 $n=5$ 時本體執行 5 次 , 條件測試 6 次。
- 3 改成 for 時 , 將初始化、條件、更新搬入表頭 , 本體不再重複 $i++$ 。

重要變數與狀態

名稱	型別 / 形式	用途
n	int	要印出的列數
i	int	目前列編號
$1LL*i*i$	long long 運算式	本列平方

固定範例追蹤

這是依指定範例整理的狀態表 ; 切換步驟只改變標記 , 不是在瀏覽器中編譯或執行 C 。

i	$i \leq 5 ?$	輸出
1	是	1 1
2	是	2 4
3	是	3 9
4	是	4 16
5	是	5 25
6	否	不輸出 , 結束

為什麼這個解法成立？

每次印出一列之後 i 恰好增加 1 , 因此合法 n 會依序產生 $1 \sim n$ 、共 n 列。while 與 for 版本的初始化、條件、本體、更新相同 ; 本例沒有 continue 。

跳至主要內容

間 $O(1)$, 不含輸出。

03 完整程式碼

每一份都可獨立編譯

本題同時提供 `while` 與 `for` 兩個完整程式，各有自己的 `main`。請分開編譯；兩版使用同一組題目與測試資料。

c06_squares_while.c

while 版本 · 15 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int n, i = 1;
    if (scanf("%d", &n) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (n < 1 || n > 20) {
        puts("ERR_RANGE"); return 1;
    }
    while (i <= n) {
        printf("%d %lld\n", i, 1LL * i * i);
        i++;
    }
    return 0;
}
```

單題編譯（在包含本檔的資料夾執行）：

```
gcc -std=c11 -Wall -Wextra -Wpedantic c06_squares_while.c -o c06_squares_while
```

c06_squares_for.c

for 版本 · 14 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int n;
    if (scanf("%d", &n) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (n < 1 || n > 20) {
        puts("ERR_RANGE"); return 1;
    }
    for (int i = 1; i <= n; i++) {
        printf("%d %lld\n", i, 1LL * i * i);
    }
    return 0;
}
```

[跳至主要內容](#)

單題編譯（在包含本檔的資料夾執行）：

```
gcc -std=c11 -Wall -Wextra -Wpedantic c06_squares_for.c -o c06_squares_for
```


04

測試資料

7 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

C06-T01

題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
5	1 1 2 4 3 9 4 16 5 25
2 bytes · 最後有 LF 換行	22 bytes · 包含程式實際輸出的換行與空白

—

C06-T02

最小列數，恰好印一行

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1	1 1
2 bytes · 最後有 LF 換行	4 bytes · 包含程式實際輸出的換行與空白

—

C06-T03

連續三列，不多一列也不少一列

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
3	1 1 2 4 3 9
跳至主要內容 : 換行	12 bytes · 包含程式實際輸出的換行與空白

—

C06-T04 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0	ERR_RANGE
2 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

—

C06-T05 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
21	ERR_RANGE
3 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

—

C06-T06 非數字 token : scanf 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
x	ERR_INPUT
2 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

—

C06-T07 最大合法列數，兩版本同測

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
20	1 1
	2 4
	3 9
	4 16
	5 25
	6 36
	7 49
跳至主要內容	

```
8 64
9 81
10 100
11 121
12 144
13 169
14 196
15 225
16 256
17 289
18 324
19 361
20 400
```

119 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- while 與 for 的對照以本體不含 continue 為前提；continue 的更新位置另行比較。
- 平方差依序為 3、5、7...，可理解原教材 square3.c 的奇數累加法。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

C07

正式案例

迴圈

C11

以 0 結束的數列加總

while

sentinel

EOF

加總

對應整合版：第 38–40 頁 · 1 份程式 · 12 組測試

來源對照：Ch6 pp.15–16：sum.c；Ch7 pp.29–31：sum2.c

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01 完整題目

先確認規格，再寫程式

連續讀入整數，累加所有非零資料，遇到 0 結束並輸出總和。也允許輸入檔在資料末尾直接結束。

輸入規格

最多 1000 筆非零整數，每筆 -1,000,000 ~ 1,000,000；
以 0 或 EOF 結束。

輸出規格

正常輸出 Sum: 總和；讀取失敗、超出範圍或超過筆數
時輸出 ERR_INPUT / ERR_RANGE。

規則與限制

- 0 是結束記號，不計入筆數；正負資料都可累加。
- 完全沒有資料時總和為 0。
- 總和絕對值最多 10^9 ，使用 long long 儲存。

輸入 / 輸出範例

STDIN · 範例輸入

8 23 71 5 0

12 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Sum: 107

9 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02 解題說明

理解狀態如何改變

！ 跳至主要內容
讀一筆整數

STEP 02

處理 0 / EOF

STEP 03

檢查筆數與範圍

STEP 04

加總後讀下一筆

- 1 每次先讀資料，再分辨 sentinel、合法資料與錯誤。
- 2 迴圈內累加前驗證範圍，總和不含 0 終止記號。
- 3 status=1、0、EOF 是三種不同結果，不把讀取失敗當成正常資料。

重要變數與狀態

名稱	型別 / 形式	用途
n / sum	long long	本筆整數 / 累計總和
count	int	已接受的非零整數筆數
status	int	scanf 回傳狀態

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

讀入	count	sum	行為
8	1	8	接受
23	2	31	接受
71	3	102	接受
5	4	107	接受
0	4	107	停止；0 不計入

為什麼這個解法成立？

每次成功接受資料後，sum 等於目前已接受資料的總和，count 等於非零合法資料筆數。0 不加入總和；正常 EOF 也保留之前的總和。

k 為非零資料筆數：時間 O(k)；額外空間 O(1)。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

c07_sum.c

完整程式 · 19 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    long long n, sum = 0;
    int count = 0, status;
    while ((status = scanf("%lld", &n)) == 1) {
        if (n == 0) break;
        if (n < -1000000LL || n > 1000000LL ||
            count == 1000) {
            puts("ERR_RANGE"); return 1;
        }
        sum += n;
        count++;
    }
    if (status == 0 || ferror(stdin)) {
        puts("ERR_INPUT"); return 1;
    }
    printf("Sum: %lld\n", sum);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c07_sum.c -o c07_sum
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

12 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— c07-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 12 input bytes · 完整比對

[跳至主要內容](#)

STDIN · 完整輸入

8 23 71 5 0

12 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Sum: 107

9 bytes · 包含程式實際輸出的換行與空白

–

C07-T02 第一筆就是停止記號

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Sum: 0

7 bytes · 包含程式實際輸出的換行與空白

–

C07-T03 正負互相抵消

exit 0 ✓

原資源包 · 7 input bytes · 完整比對

STDIN · 完整輸入

5 -5 0

7 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Sum: 0

7 bytes · 包含程式實際輸出的換行與空白

–

C07-T04 無結尾換行 · 以 EOF 結束

exit 0 ✓

原資源包 · 5 input bytes · 完整比對

STDIN · 完整輸入

1 2 3

5 bytes · 最後沒有換行 · 檔案直接結束

STDOUT · 完整預期輸出

Sum: 6

7 bytes · 包含程式實際輸出的換行與空白

–

跳至主要內容 · 檔：立即 EOF

exit 0 ✓

🔗 · 0 input bytes · 完整比對

STDIN · 完整輸入

(空輸入檔，直接 EOF)

0 bytes · 0 bytes · 不是空白行

STDOUT · 完整預期輸出

Sum: 0

7 bytes · 包含程式實際輸出的換行與空白

— C07-T06 驗證 ERR_INPUT

```
exit 1 ✓
```

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

1 x

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

— C07-T07 驗證 ERR_RANGE

```
exit 1 ✓
```

原資源包 · 10 input bytes · 完整比對

STDIN · 完整輸入

1000001 0

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

- C07-T08 1001 筆非零值：超過 1000 筆上限

```
exit 1 ✓
```

原資源包上限測試 · 2,004 input bytes · 完整比對

STDIN · 完整輸入

1 1

2,004 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

```
exit 1 ✓
```

L 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入
x
2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

- C07-T10

1000 筆最大值，總和為 10^9

exit 0 ✓
- 本 HTML 新增測試 · 8,002 input bytes · 完整比對

STDIN · 完整輸入
1000000 1000000 1000000 1000000 1000000 1000000 10000
8,002 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Sum: 1000000000
16 bytes · 包含程式實際輸出的換行與空白

- C07-T11

低於最小合法值

exit 1 ✓
- 本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入
-1000001
9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

- C07-T12

停止記號後的資料不再讀取

exit 0 ✓
- 本 HTML 新增測試 · 8 input bytes · 完整比對

STDIN · 完整輸入
1 0 999
8 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Sum: 1
7 bytes · 包含程式實際輸出的換行與空白

錯誤 跳至主要內容 也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- `sum` 必須初始化為 0，`scanf` 的型別必須與 `long long` 對應。
- 本版修正原教材 `for` 改寫中格式不一致及未檢查輸入的問題。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

計算十進位位數

do while

位數

整除

對應整合版：第 41–43 頁 · 1 份程式 · 9 組測試

來源對照：Ch6 pp.21–22：numdigits.c

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

輸入一個非負整數，以反覆整除 10 的方式計算其十進位位數；不得把數字轉成字串處理。

輸入規格

一個整數 n ， $0 \sim 1,000,000,000$ 。

輸出規格

輸出 Digits: 位數；錯誤輸出 ERR_INPUT / ERR_RANGE。

規則與限制

- 0 的位數定義為 1。
- 輸入解讀為數值：例如 00060 與 60 都是 2 位數。
- 每次 $n/=10$ 去除最低一位，至少執行一次。

輸入 / 輸出範例

STDIN · 範例輸入

60

3 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Digits: 2

10 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



- 1 do 的本體先執行，因此 $n=0$ 仍可讓 `digits` 加 1。
- 2 以 60 追蹤： $60 \rightarrow 6 \rightarrow 0$ ，共兩次整除。
- 3 程式會改變 `n`；若還要印出原輸入，需先另存副本。

重要變數與狀態

名稱	型別 / 形式	用途
<code>n</code>	<code>long long</code>	尚未移除的數位部分
<code>digits</code>	<code>int</code>	已完成的整除次數

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

回合	除法前 <code>n</code>	除法後 <code>n</code>	<code>digits</code>
1	60	6	1
2	6	0	2
終止	0	不再執行	Digits: 2

為什麼這個解法成立？

每次整除 10 去除一個十進位數位。`do...while` 保證至少執行一次，故 0 也會得到 1 位。輸入的前導 0 不屬於數值本身。

`d` 為十進位位數：時間 $O(d)$ ；額外空間 $O(1)$ 。

03 完整程式碼

每一份都可獨立編譯

c08_digits.c
完整程式 · 17 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    long long n;
    int digits = 0;

    跳至主要內容
    while (scanf("%lld", &n) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (n < 0 || n > 1000000000LL) {
```

```
        puts("ERR_RANGE"); return 1;
    }
    do {
        n /= 10;
        digits++;
    } while (n > 0);
    printf("Digits: %d\n", digits);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c08_digits.c -o c08_digits
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

9 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C08-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
60	Digits: 2
3 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

— C08-T02 0仍有一位，須至少執行一次

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0	Digits: 1
2 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

— C08-T03 個位數上界

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

9

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Digits: 1

10 bytes · 包含程式實際輸出的換行與空白

- C08-T04 跨過10的位數門檻

exit 0 ✓
- 原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

10

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Digits: 2

10 bytes · 包含程式實際輸出的換行與空白

- C08-T05 最大值有10位

exit 0 ✓
- 原資源包 · 11 input bytes · 完整比對

STDIN · 完整輸入

1000000000

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Digits: 10

11 bytes · 包含程式實際輸出的換行與空白

- C08-T06 驗證 ERR_RANGE

exit 1 ✓
- 原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容

token : scanf 讀取失敗

exit 1 ✓
- L 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入
x
2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

–

C08-T08 前導 0 不增加數值的位數

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入
00060
6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Digits: 2
10 bytes · 包含程式實際輸出的換行與空白

–

C08-T09 數值超出上限

exit 1 ✓

本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入
1000000001
11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 不要把 while(n>0) 原封不動換進來，否則 0 會得到 0 位數。
- do...while 結尾必須有分號，和一般 while 本體前的多餘分號不同。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制與測試規則。

跳至主要內容

質數判定與提早停止

for

break

質數

平方根

對應整合版：第 45–47 頁 · 1 份程式 · 11 組測試

來源對照：Ch6 pp.48–49：prime / break

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

判斷整數 n 是否為質數。若不是質數且 $n \geq 2$ ，找出最小的整除因子；找到第一個因子後立即停止搜尋。

輸入規格

一個整數 n ， $0 \sim 1,000,000,000$ 。

輸出規格

質數輸出 PRIME； $n < 2$ 輸出 NOT_PRIME；其他合數輸出 DIVISOR: 最小因子。

規則與限制

- 質數必須大於或等於 2。
- 從 $d=2$ 依序測試；只需測到平方根附近。
- 以 $d \leq n/d$ 取代 $d * d \leq n$ ，避免乘積中途超出型別範圍。

輸入 / 輸出範例

STDIN · 範例輸入

91

3 bytes · 最後有 LF 換行

STDOUT · 範例輸出

DIVISOR: 7

11 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變

！ 跳至主要內容

排除 $n < 2$

STEP 02

d 從 2 出發

STEP 03

測試 $n \% d$

STEP 04

找到就 break，否則續查

核心思考與步驟

- 1
- 若 $n=a \times b$ 且兩個因子都大於 $\sqrt{n}$ ，乘積會大於 n ，矛盾。
- 2
- 由小到大測試，所以第一個找到的因子必然最小。
- 3
- `divisor=0` 表示尚未找到；`break` 只離開 `for`，不是整個 `main`。

重要變數與狀態

名稱	型別 / 形式	用途
<code>n</code>	<code>long long</code>	待判定數
<code>d</code>	<code>long long</code>	目前測試的除數
<code>divisor</code>	<code>long long</code>	0 代表尚未找到因子

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

<code>d</code>	<code>d ≤ 91/d ?</code>	<code>91 % d</code>	行為
2	是	1	續查
3	是	1	續查
4	是	3	續查
5	是	1	續查
6	是	1	續查
7	是	0	<code>divisor=7 ; break</code>

為什麼這個解法成立？

若合數 n 的兩個因子都大於 $\sqrt{n}$ ，其乘積會大於 n ，矛盾。由 2 遞增測試至 $\sqrt{n}$ 的整數因子，第一個找到的必為最小因子；沒有找到則為質數。

最壞時間 $O(\sqrt{n})$ ；額外空間 $O(1)$ 。

跳至主要內容

0 碼

每一份都可獨立編譯

```
c09_prime.c
```

完整程式 · 20 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    long long n, divisor = 0;
    if (scanf("%lld", &n) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (n < 0 || n > 1000000000LL) {
        puts("ERR_RANGE"); return 1;
    }
    if (n < 2) { puts("NOT_PRIME"); return 0; }
    for (long long d = 2; d <= n / d; d++) {
        if (n % d == 0) {
            divisor = d;
            break;
        }
    }
    if (divisor == 0) puts("PRIME");
    else printf("DIVISOR: %lld\n", divisor);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c09_prime.c -o c09_prime
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

11 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C09-T01

題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
91	DIVISOR: 7
3 bytes · 最後有 LF 換行	11 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

— C09-T02

0不是質數

exit 0 ✓

原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

NOT_PRIME

10 bytes · 包含程式實際輸出的換行與空白

- C09-T03

1不是質數

exit 0 ✓
- 原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

1

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

NOT_PRIME

10 bytes · 包含程式實際輸出的換行與空白

- C09-T04

最小質數2

exit 0 ✓
- 原資源包 · 2 input bytes · 完整比對

STDIN · 完整輸入

2

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

PRIME

6 bytes · 包含程式實際輸出的換行與空白

- C09-T05

平方數49 · 因子7在搜尋上界

exit 0 ✓
- 原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

49

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

DIVISOR: 7

11 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容

質數97

exit 0 ✓
- 3 · 3 input bytes · 完整比對

STDIN · 完整輸入

97

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

PRIME

6 bytes · 包含程式實際輸出的換行與空白

- C09-T07 找到最小因子2後提早停止

exit 0 ✓
- 原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

100

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

DIVISOR: 2

11 bytes · 包含程式實際輸出的換行與空白

- C09-T08 非數字 token：scanf 讀取失敗

exit 1 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

- C09-T09 最大合法 n，第一輪即找到因子

exit 0 ✓
- 本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入

1000000000

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

DIVISOR: 2

11 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容

!：需走完平方根範圍

exit 0 ✓
- L 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入
999999937
10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
PRIME
6 bytes · 包含程式實際輸出的換行與空白

– C09-T11 負值不符合題目範圍

exit 1 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入
-1
3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- n=2 是測試重點：迴圈可以 0 次，答案仍為 PRIME。
- 原教材逐一試到 n-1；本版增加平方根上界並處理 n<2。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

座位編號產生器

巢狀for

座位

格式

對應整合版：第 48–50 頁 · 1 份程式 · 8 組測試

來源對照：延伸 Ch6 的 for 與巢狀控制結構

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

為活動會場建立矩形座位表。輸入列數 rows 與每列座位數 cols，以「列-座號」格式逐列輸出。

輸入規格

兩個整數 rows、cols，均為 1 ~ 10。

輸出規格

每個座位格式 r-c；同列相鄰座位間一個空白，每列結尾換行，行尾沒有多餘空白。

規則與限制

- 列、座號都從 1 開始。
- 外層控制列，內層控制該列各座位。
- 無效輸入輸出 ERR_INPUT 或 ERR_RANGE。

輸入 / 輸出範例

STDIN · 範例輸入

2 3

4 bytes · 最後有 LF 換行

STDOUT · 範例輸出

1-1 1-2 1-3
2-1 2-2 2-3

24 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變

跳至主要內容
r 從 1 到 rows

STEP 02
c 從 1 到 cols

STEP 03
輸出 r-c 與間隔

STEP 04
整列完成才換行

- 1 固定 `r`，讓 `c` 由 1 跑到 `cols`；完成整列才換行。
- 2 每次進入新的一列，都由內層 `for` 重新初始化 `c=1`。
- 3 `2×3` 的座位列印動作共 6 次，換行 2 次。

重要變數與狀態

名稱	型別 / 形式	用途
<code>rows / cols</code>	<code>int</code>	總列數 / 每列座位數
<code>r / c</code>	<code>int</code>	目前列 / 座位

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

<code>r</code>	<code>c</code>	輸出片段	行尾處理
1	1	1-1	加空白
1	2	1-2	加空白
1	3	1-3	換行
2	1	2-1	加空白
2	2	2-2	加空白
2	3	2-3	換行

為什麼這個解法成立？

外層對每一列執行一次，內層每次重新由 1 開始，產生所有 `rows×cols` 個座位。只有 `c<cols` 才印空白，因此沒有尾隨空白。

時間 $O(\text{rows} \times \text{cols})$ ；額外空間 $O(1)$ ，不含輸出。

[跳至主要內容](#)

03 完整程式碼

每一份都可獨立編譯

c10_seats.c

完整程式 · 19 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int rows, cols;
    if (scanf("%d %d", &rows, &cols) != 2) {
        puts("ERR_INPUT"); return 1;
    }
    if (rows < 1 || rows > 10 ||
        cols < 1 || cols > 10) {
        puts("ERR_RANGE"); return 1;
    }
    for (int r = 1; r <= rows; r++) {
        for (int c = 1; c <= cols; c++) {
            printf("%d-%d", r, c);
            if (c < cols) putchar(' ');
        }
        putchar('\n');
    }
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c10_seats.c -o c10_seats
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

8 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C10-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

[跳至主要內容](#)

STDIN · 完整輸入

2 3

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

1-1 1-2 1-3
2-1 2-2 2-3

24 bytes · 包含程式實際輸出的換行與空白

- C10-T02 只有一席 · 不印多餘空白

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

1 1

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

1-1

4 bytes · 包含程式實際輸出的換行與空白

- C10-T03 多列單欄 · 每列均重新開始

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

3 1

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

1-1
2-1
3-1

12 bytes · 包含程式實際輸出的換行與空白

- C10-T04 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

0 3

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

- C10-T05 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 5 input bytes · 完整比對

STDIN · 完整輸入
11 3
5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

- C10-T06 非數字 token：scanf 讀取失敗

exit 1 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入
x
2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

- C10-T07 座位數不得為 0

exit 1 ✓
- 本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入
2 0
4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

- C10-T08 最大 10×10 座位表及空白格式

exit 0 ✓
- 本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入
10 10
6 bytes · 最後有 LF 換行
跳至主要內容

STDOUT · 完整預期輸出
1-1 1-2 1-3 1-4 1-5 1-6 1-7 1-8 1-9 1-10 2-1 2-2 2-3 2-4 2-5 2-6 2-7 2-8 2-9 2-10 3-1 3-2 3-3 3-4 3-5 3-6 3-7 3-8 3-9 3-10 4-1 4-2 4-3 4-4 4-5 4-6 4-7 4-8 4-9 4-10 5-1 5-2 5-3 5-4 5-5 5-6 5-7 5-8 5-9 5-10 6-1 6-2 6-3 6-4 6-5 6-6 6-7 6-8 6-9 6-10 7-1 7-2 7-3 7-4 7-5 7-6 7-7 7-8 7-9 7-10 8-1 8-2 8-3 8-4 8-5 8-6 8-7 8-8 8-9 8-10 9-1 9-2 9-3 9-4 9-5 9-6 9-7 9-8 9-9 9-10 10-1 10-2 10-3 10-4 10-5 10-6 10-7 10-8 10-9 10-10

420 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 把換行放進內層，會讓每個座位各佔一行。
- 把 c 的初始化放到外層之外，第二列可能完全不執行。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

距離感測資料清理

continue

break

資料清理

平均

對應整合版：第 51–53 頁 · 1 份程式 · 11 組測試

來源對照：Ch6 pp.50–54：break / continue；Ch7 pp.92–95：cast

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

連續讀入距離感測值，忽略超出有效量測範圍的資料。收到停止記號後，輸出有效資料筆數及平均距離。此為教學模擬，不是設備規格。

輸入規格

整數串流，以 -999 或 EOF 結束；最多處理 1000 筆非停止記號資料。

輸出規格

有有效值輸出 Count: 筆數與 Mean: 平均 (2 位小數) ；沒有有效值輸出 NO_DATA。

- 規則與限制
- 0 ~ 10000 (毫米) 為有效值；其他值略過但仍算已處理一筆。
 - 999 是停止記號，需在範圍篩選前檢查。
 - 超過 1000 筆輸出 ERR_LIMIT；非整數資料輸出 ERR_INPUT。

輸入 / 輸出範例

STDIN · 範例輸入

100 120 -5 140 -999

20 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Count: 3
Mean: 120.00

22 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

0

跳至主要內容

理解狀態如何改變

STEP 01

STEP 02

STEP 03

STEP 04

核心思考與步驟

- 1 停止、總筆數限制、有效性篩選必須依序處理。
- 2 continue 略過 sum/count 更新，但下一輪仍會讀取新資料。
- 3 count=0 時不能算平均；平均須先將 sum 轉成 double。

重要變數與狀態

名稱	型別 / 形式	用途
x / sum	long long	本筆讀值 / 有效總和
seen	int	非停止記號總筆數
count	int	有效讀值筆數
status	int	讀取狀態

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

讀值	seen	count	sum	行為
100	1	1	100	接受
120	2	2	220	接受
-5	3	2	220	continue
140	4	3	360	接受
-999	4	3	360	break ; Mean=120.00

為什麼這個解法成立？

seen 包含無效讀值，count 與 sum 只反映合法距離資料。停止記號先判斷，否則它會被當成普通無效值而無法停止。count=0 時不做除法。

跳至主要內容

O(k)；額外空間 O(1)。

03 完整程式碼

每一份都可獨立編譯

c11_sensor.c

完整程式 · 22 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    long long x, sum = 0;
    int count = 0, seen = 0, status;
    while ((status = scanf("%lld", &x)) == 1) {
        if (x == -999) break;
        if (seen == 1000) {
            puts("ERR_LIMIT"); return 1;
        }
        seen++;
        if (x < 0 || x > 10000) continue;
        sum += x;
        count++;
    }
    if (status == 0 || ferror(stdin)) {
        puts("ERR_INPUT"); return 1;
    }
    if (count == 0) { puts("NO_DATA"); return 0; }
    printf("Count: %d\n", count);
    printf("Mean: %.2f\n", (double)sum / count);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c11_sensor.c -o c11_sensor
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

11 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C11-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 20 input bytes · 完整比對

跳至主要內容

STDOUT · 完整預期輸出

```
100 120 -5 140 -999
```

```
Count: 3
```

20 bytes · 最後有 LF 換行

Mean: 120.00
22 bytes · 包含程式實際輸出的換行與空白

- C11-T02 停止記號前沒有有效資料

exit 0 ✓
- 原資源包 · 5 input bytes · 完整比對

STDIN · 完整輸入
-999
5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
NO_DATA
8 bytes · 包含程式實際輸出的換行與空白

- C11-T03 所有讀值都無效

exit 0 ✓
- 原資源包 · 14 input bytes · 完整比對

STDIN · 完整輸入
-5 10001 -999
14 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
NO_DATA
8 bytes · 包含程式實際輸出的換行與空白

- C11-T04 有效範圍的下界0及上界10000

exit 0 ✓
- 原資源包 · 13 input bytes · 完整比對

STDIN · 完整輸入
0 10000 -999
13 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
Count: 2 Mean: 5000.00
23 bytes · 包含程式實際輸出的換行與空白

- C11-T05 平均值應保留小數

exit 0 ✓
- 原資源包 · 9 input bytes · 完整比對

跳至主要內容
1 2 -999

STDOUT · 完整預期輸出
Count: 2 Mean: 1.50

9 bytes · 最後有 LF 換行

20 bytes · 包含程式實際輸出的換行與空白

- C11-T06 驗證 ERR_INPUT

exit 1 ✓
- 原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

1 x

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

- C11-T07 1001 筆無效值仍須計入 seen 上限

exit 1 ✓
- 原資源包上限測試 · 3,008 input bytes · 完整比對

STDIN · 完整輸入

-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1

3,008 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_LIMIT

10 bytes · 包含程式實際輸出的換行與空白

- C11-T08 非數字 token : scanf 讀取失敗

exit 1 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

- C11-T09 空檔立即 EOF

exit 0 ✓
- 本 HTML 新增測試 · 0 input bytes · 完整比對

跳至主要內容

量測目前環境的型別

sizeof

limits.h

float.h

環境觀察

對應整合版：第 63–65 頁 · 1 份程式 · 1 組測試

來源對照：Ch7 pp.6–15、39、103–106：limits / float.h / sizeof

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01 完整題目

先確認規格，再寫程式

不要背誦「64 位元電腦的 long 一定是 8 bytes」。撰寫不需輸入的實驗程式，查詢並輸出實際編譯環境的型別大小、整數範圍及浮點特性。

輸入規格

不需輸入；在目前的 C11 編譯環境直接執行。

輸出規格

依程式標籤列印 CHAR_BIT、四種型別大小、int/long 範圍、FLT_DIG、DBL_DIG、FLT_MIN 與 FLT_TRUE_MIN。

規則與限制

- 大小單位是 C byte；一個 byte 有 CHAR_BIT bits。
- 使用標頭提供的巨集，不能硬寫平台數值。
- 輸出可因平台而異；右側只提供本次測試環境的觀察，不是標準保證。

輸入 / 輸出範例

以下輸出為本次 Linux / GCC 的完整觀察值，不是所有電腦都必須相同的固定答案。本題由 <limits.h>、<float.h> 與 sizeof 取得環境值。驗證欄位齊全、數值可解析、大小 / 範圍基本關係合理；不要求其他平台和這裡的 Linux 輸出相同。

STDIN · 範例輸入

(無輸入)

15 bytes · 最後沒有換行 · 檔案直接結束

跳至主要內容

STDOUT · 本次觀察

```
CHAR_BIT=8
sizeof(int)=4
sizeof(long)=8
sizeof(long long)=8
sizeof(double)=8
INT_MIN=-2147483648 INT_MAX=2147483647
LONG_MIN=-9223372036854775808 LONG_MAX=9223372036854775807
FLT_DIG=6 DBL_DIG=15
```

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1
- sizeof 告訴你儲存空間，limits.h 告訴你可表示範圍，兩者不要混為一談。
- 2
- 用 %zu 輸出 sizeof 的結果；用正確格式輸出其他巨集。
- 3
- 比較兩台電腦的結果，觀察 long 及浮點參數，不以 CPU 位元數直接推測。

重要變數與狀態

名稱	型別 / 形式	用途
CHAR_BIT	標頭巨集	每個 C byte 的位元數
sizeof(...)	size_t 結果	型別佔用的 C bytes
INT_MIN / LONG_MAX 等	標頭巨集	目前環境實際範圍

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

查詢類型	程式使用方式	解讀
大小	sizeof(long)	不要由 CPU 位元數推測
整數範圍	LONG_MIN / LONG_MAX	由實作提供
十進位精度	FLT_DIG / DBL_DIG	不是小數點後位數
 跳至主要內容	FLT_MIN / FLT_TRUE_MIN	正常數與所有正數下界不同

為什麼這個解法成立？

程式沒有把平台值寫死，而是從編譯器與標頭取得。測試核對必要欄位與成功執行，不把本次觀察值當成所有平台必須符合的固定答案。

固定數量的查詢：時間 $O(1)$ ；額外空間 $O(1)$ 。

03 完整程式碼

每一份都可獨立編譯

c12_type_lab.c

完整程式 · 16 行 · 長行可橫向捲動

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
int main(void) {
    printf("CHAR_BIT=%d\n", CHAR_BIT);
    printf("sizeof(int)=%zu\n", sizeof(int));
    printf("sizeof(long)=%zu\n", sizeof(long));
    printf("sizeof(long long)=%zu\n", sizeof(long long));
    printf("sizeof(double)=%zu\n", sizeof(double));
    printf("INT_MIN=%d INT_MAX=%d\n", INT_MIN, INT_MAX);
    printf("LONG_MIN=%ld LONG_MAX=%ld\n", LONG_MIN, LONG_MAX);
    printf("FLT_DIG=%d DBL_DIG=%d\n", FLT_DIG, DBL_DIG);
    printf("FLT_MIN=%e\n", (double)FLT_MIN);
    printf("FLT_TRUE_MIN=%e\n", (double)FLT_TRUE_MIN);
    return 0;
}
```

單題編譯（在包含本檔的資料夾執行）：

```
gcc -std=c11 -Wall -Wextra -Wpedantic c12_type_lab.c -o c12_type_lab
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

1 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

本題由 `<limits.h>`、`<float.h>` 與 `sizeof` 取得環境值。驗證欄位齊全、數值可解析、大小 / 範圍基本關係合理；不要求其
他平台和這裡的 Linux 輸出相同。

[跳至主要內容](#)

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 0 bytes · 不是空白行

STDOUT · 本次觀察（非跨平台固定答案）

CHAR_BIT=8
sizeof(int)=4
sizeof(long)=8
sizeof(long long)=8
sizeof(double)=8
INT_MIN=-2147483648 INT_MAX=2147483647
LONG_MIN=-9223372036854775808 LONG_MAX=9223372036854775807
FLT_DIG=6 DBL_DIG=15
FLT_MIN=1.175494e-38
FLT_TRUE_MIN=1.401298e-45

243 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- FLT_TRUE_MIN 是 C11 巨集；因此本份實作統一以 -std=c11 編譯。
- 實驗不量測指標與複合型別，以免超出本章核心。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

RGB 影片的記憶體估算

cast

overflow

RGB

MiB

對應整合版：第 71–73 頁 · 1 份程式 · 8 組測試

來源對照：Ch7 pp.25–26、92–96：overflow / cast

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

影像系統要將多張 RGB 影格同時存入記憶體。每個像素固定佔 3 bytes，請計算所有未壓縮影格所需的 bytes 與 MiB。

輸入規格

整數 width、height、frames；寬高各 1~20000，影格數 1~3600。

輸出規格

輸出 Bytes: 整數容量，以及 MiB: 容量（2 位小數）；錯誤輸出 ERR_INPUT / ERR_RANGE。

規則與限制

- bytes = width × height × 3 × frames。
- 1 MiB = 1024×1024 bytes；不含列對齊、metadata、壓縮或額外副本。
- 使用 long long 進行乘法；最大 4.32×10^12，可由至少 64 位的 long long 容納。

輸入 / 輸出範例

STDIN · 範例輸入

1920 1080 600

14 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Bytes: 3732480000
MiB: 3559.57

31 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變

跳至主要內容
驗證三個維度

STEP 02
先轉 long long

STEP 03
乘出總 bytes

STEP 04
以浮點除法算 MiB

核心思考與步驟

- 1
- 先推導最大結果，再選型別；不要等溢位後才補救。
- 2
- 第一個乘法前就把 width 轉成 long long，後續整數乘法沿用較寬型別。
- 3
- bytes / 1048576 若都是整數會先截斷；轉成 double 後再除。

重要變數與狀態

名稱	型別 / 形式	用途
width / height / frames	int	寬、高與影格數
bytes	long long	全部像素資料 bytes
mib	double	換算後的 MiB

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

階段	運算	結果
像素 / 張	1920 × 1080	2,073,600
bytes / 張	2,073,600 × 3	6,220,800
全部影格	6,220,800 × 600	3,732,480,000
MiB	3,732,480,000 ÷ 1,048,576	3559.57 (顯示到 2 位)

為什麼這個解法成立？

合法輸入的最大乘積是 20000×20000×3×3600 = 4.32×10^12。第一個運算元已轉成 long long，避免先在 int 中溢位；換算 MiB 前再轉成 double，保留小數部分。

時間 O(1)；額外空間 O(1)。

03

完整程式碼

每一份都可獨立編譯

跳至主要內容

```
#include <stdio.h>
int main(void) {
```

```
int width, height, frames;
if (scanf("%d %d %d", &width, &height, &frames) != 3) {
    puts("ERR_INPUT"); return 1;
}
if (width < 1 || width > 20000 ||
    height < 1 || height > 20000 ||
    frames < 1 || frames > 3600) {
    puts("ERR_RANGE"); return 1;
}
long long bytes = (long long)width * height * 3 * frames;
double mib = (double)bytes / (1024.0 * 1024.0);
printf("Bytes: %lld\n", bytes);
printf("MiB: %.2f\n", mib);
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c13_rgb_memory.c -o c13_rgb_memory
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

8 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

- C13-T01

題目示範輸入 / 輸出

exit 0 ✓
- 原資源包 · 14 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1920 1080 600	Bytes: 3732480000 MiB: 3559.57
14 bytes · 最後有 LF 換行	31 bytes · 包含程式實際輸出的換行與空白

- C13-T02

最小影格共3bytes · MiB四捨五入顯示0.00

exit 0 ✓
- 原資源包 · 6 input bytes · 完整比對

跳至主要內容	STDOUT · 完整預期輸出
1 1 1	Bytes: 3 MiB: 0.00

6 bytes · 最後有 LF 換行

19 bytes · 包含程式實際輸出的換行與空白

—

C13-T03 三個維度均在上限 · 驗證大乘積

exit 0 ✓

原資源包 · 17 input bytes · 完整比對

STDIN · 完整輸入

20000 20000 3600

17 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Bytes: 4320000000000
MiB: 4119873.05

37 bytes · 包含程式實際輸出的換行與空白

—

C13-T04 驗證 ERR_RANGE

exit 1 ✓

原資源包 · 11 input bytes · 完整比對

STDIN · 完整輸入

0 1080 600

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

—

C13-T05 非數字 token : scanf 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

—

C13-T06 寬度超限

exit 1 ✓

本 HTML 新增測試 · 10 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入
20001 1 1
10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

— C13-T07 高度超限

exit 1 ✓

本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入
1 20001 1
10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

— C13-T08 影格數超限

exit 1 ✓

本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入
1 1 3601
9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_RANGE
10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- (long long)(width*height*3*frames) 轉型太晚；指定結果變數型別也不會改變右側的中途型別。
- 這是原始像素資料量，不是影片檔案大小或 GPU 可用容量估計。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制與測試規則。

跳至主要內容

一行文字的分類統計

getchar

ctype

EOF

ASCII

對應整合版：第 76–79 頁 · 1 份程式 · 9 組測試

來源對照：Ch7 pp.60–73：ctype / getchar / length.c

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

讀取一行 ASCII 文字，統計總字元數、英文字母數、數字數及空白字元數。必須以 `getchar` 逐字讀取，不使用字串陣列。

輸入規格

一行 ASCII 文字，最多 4096 bytes；遇換行或 EOF 結束。

輸出規格

輸出 Length、Letters、Digits、Spaces 四個計數；超長輸出 ERR_LIMIT；串流讀取錯誤輸出 ERR_INPUT。

- 規則與限制
- 總長度包含標點與空白，不包含結尾的換行。
 - 空白類別使用 `isspace`，包含空白與 `tab`；分類以預設 C locale 為準。
 - 題目限定 ASCII；本程式不把 UTF-8 中文的多個 bytes 合併成一個字。

輸入 / 輸出範例

STDIN · 範例輸入

Ab 12!

7 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Length: 6
Letters: 2
Digits: 2
Spaces: 1

41 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。



核心思考與步驟

- 1
- 用 int 接收 getchar，先排除 EOF 與換行，再更新計數。
- 2
- 每個有效讀入 byte 都增加 len，標點即使不屬於三種類別仍計入總長度。
- 3
- getchar 成功值可直接交給 ctype；若原值來自 char 變數，先轉 unsigned char。

重要變數與狀態

名稱	型別 / 形式	用途
ch	int	目前 byte 或 EOF
len	size_t	總 byte 數
letters / digits / spaces	size_t	各分類計數

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

字元	len	letters	digits	spaces
A	1	1	0	0
b	2	2	0	0
空白	3	2	0	1
1	4	2	1	1
2	5	2	2	1
!	6	2	2	1
換行	6	2	2	1

為什麼這個解法成立？

所有非換行的成功讀值都恰好增加 len 一次；分類只增加相應計數，標點仍計入總長度。先排除 EOF，再使用 ctype。題目限定 ASCII，統計不是 Unicode 字數統計。

跳至主要內容

L 為該行 byte 數：時間 O(L)；額外空間 O(1)。

c14_text_stats.c
完整程式 · 19 行 · 長行可橫向捲動

```
#include <stdio.h>
#include <ctype.h>
int main(void) {
    int ch;
    size_t len = 0, letters = 0, digits = 0, spaces = 0;
    while ((ch = getchar()) != EOF && ch != '\n') {
        if (len == 4096) { puts("ERR_LIMIT"); return 1; }
        len++;
        if (isalpha(ch)) letters++;
        else if (isdigit(ch)) digits++;
        else if (isspace(ch)) spaces++;
    }
    if (ferror(stdin)) { puts("ERR_INPUT"); return 1; }
    printf("Length: %zu\n", len);
    printf("Letters: %zu\n", letters);
    printf("Digits: %zu\n", digits);
    printf("Spaces: %zu\n", spaces);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic c14_text_stats.c -o c14_text_stats
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— C14-T01 題目示範輸入 / 輸出

原資源包 · 7 input bytes · 完整比對

exit 0 ✓

STDIN · 完整輸入	STDOUT · 完整預期輸出
跳至主要內容 7 bytes · 最後有 LF 換行	Length: 6 Letters: 2 Digits: 2 Spaces: 1

41 bytes · 包含程式實際輸出的換行與空白

C14-T02 只含換行的空行

exit 0 ✓

原資源包 · 1 input bytes · 完整比對

STDIN · 完整輸入

1 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Length: 0
Letters: 0
Digits: 0
Spaces: 0

41 bytes · 包含程式實際輸出的換行與空白

C14-T03 空輸入檔：立即 EOF

exit 0 ✓

原資源包 · 0 input bytes · 完整比對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 0 bytes · 不是空白行

STDOUT · 完整預期輸出

Length: 0
Letters: 0
Digits: 0
Spaces: 0

41 bytes · 包含程式實際輸出的換行與空白

C14-T04 包含 tab 的空白分類

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

a 9

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Length: 3
Letters: 1
Digits: 1
Spaces: 1

41 bytes · 包含程式實際輸出的換行與空白

跳至主要內容 換行，以 EOF 結束

exit 0 ✓

原資源包 · 3 input bytes · 完整比對

STDIN · 完整輸入

Hi!

3 bytes · 最後沒有換行 · 檔案直接結束

STDOUT · 完整預期輸出

Length: 3
Letters: 2
Digits: 0
Spaces: 0

41 bytes · 包含程式實際輸出的換行與空白

–

C14-T06

4096 bytes : 合法長度上限

exit 0 ✓

原資源包上限測試 · 4,097 input bytes · 完整比對

STDIN · 完整輸入

aa

4,097 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Length: 4096
Letters: 4096
Digits: 0
Spaces: 0

47 bytes · 包含程式實際輸出的換行與空白

–

C14-T07

4097 bytes : 超過長度上限

exit 1 ✓

原資源包上限測試 · 4,098 input bytes · 完整比對

STDIN · 完整輸入

aa

4,098 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_LIMIT

10 bytes · 包含程式實際輸出的換行與空白

–

C14-T08

只統計第一行

exit 0 ✓

本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入

Hi!

跳至主要內容

9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Length: 3
Letters: 2
Digits: 0
Spaces: 0

41 bytes · 包含程式實際輸出的換行與空白

— C14-T09 標點與多種空白

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入

!?

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Length: 5
Letters: 0
Digits: 0
Spaces: 3

41 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- 原始 length.c 未處理 EOF；本版補上 EOF 與型別問題。
- 空白和標點也佔長度；letters+digits+spaces 不必等於 len。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

互動式零用金帳本

switch

for

狀態

整數分

對應整合版：第 83–87 頁 · 1 份程式 · 15 組測試

來源對照：Ch6 pp.59–64：checking.c；Ch7：型別與溢位

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01

完整題目

先確認規格，再寫程式

建立可反覆輸入指令的教學帳本，支援清零、收入、支出、查詢與離開。餘額以最小單位「分」存成整數，拒絕非法交易且不改變原餘額。

輸入規格

整數指令：0 清零、1 收入、2 支出、3 查詢、4 離開。

1/2 後另讀入正整數金額（分）。

輸出規格

查詢輸出 Balance: 元.分；錯誤指令 ERR_CMD；非法金額 ERR_AMOUNT；收入超限 ERR_LIMIT；餘額不足 ERR_FUNDS。

規則與限制

- 初始餘額為 0；餘額上限 1,000,000,000,000 分。
- 0 清零後輸出 CLEARED；1/2 成功輸出 OK；4 輸出 BYE。
- EOF（在指令位置）正常離開；讀取錯誤或缺少金額輸出 ERR_INPUT 並結束。
- 拒絕交易後繼續接收指令；所有金額輸入都是整數「分」，不接受小數「元」。

輸入 / 輸出範例

STDIN · 範例輸入

```
1 104256
2 13379
3
4
```

21 bytes · 最後有 LF 換行

STDOUT · 範例輸出

```
OK
OK
Balance: 908.77
BYE
```

26 bytes · 包含程式實際輸出的換行與空白

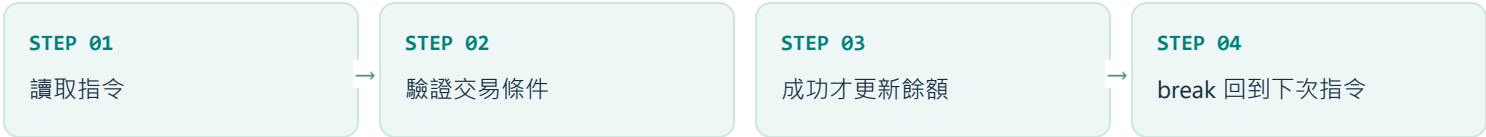
跳至主要內容

輸，²。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 外層 for(;;) 管理互動生命週期；switch 只管理單次指令。
- 2 收入先檢查剩餘容量；支出先檢查餘額。通過檢查才更新 balance。
- 3 case 內的 break 只離開 switch；4 用 return 直接結束 main。
- 4 整數分除以 100 取得元，餘數以 %02lld 顯示兩位數。

重要變數與狀態

名稱	型別 / 形式	用途
balance / amount	long long	餘額 / 交易金額，單位分
LIMIT	const long long	餘額上限
cmd / status	int	指令 / 讀取狀態

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

指令	操作前餘額 (分)	操作後餘額 (分)	輸出
1 104256	0	104256	OK
2 13379	104256	90877	OK
3	90877	90877	Balance: 908.77
4	90877	90877	BYE；結束

為什麼這個解法成立？

初始 $0 < balance < LIMIT$ 。收入先檢查 $amount \leq LIMIT - balance$ ，支出先檢查 $amount \leq balance$ ，因此接受交易後仍保持此範圍。跳至主要內容 ¶ balance。

k 為指令數：時間 $O(k)$ ；額外空間 $O(1)$ 。

03 完整程式碼

每一份都可獨立編譯

c15_ledger.c

完整程式 · 43 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    const long long LIMIT = 1000000000000LL;
    long long balance = 0, amount;
    int cmd, status;
    for (;;) {
        status = scanf("%d", &cmd);
        if (status == EOF && !ferror(stdin)) break;
        if (status != 1) { puts("ERR_INPUT"); return 1; }
        switch (cmd) {
            case 0:
                balance = 0; puts("CLEARED"); break;
            case 1: case 2:
                if (scanf("%lld", &amount) != 1) {
                    puts("ERR_INPUT"); return 1;
                }
                if (amount <= 0 || amount > LIMIT) {
                    puts("ERR_AMOUNT"); break;
                }
                if (cmd == 1) {
                    if (amount > LIMIT - balance) {
                        puts("ERR_LIMIT"); break;
                    }
                    balance += amount;
                } else {
                    if (amount > balance) {
                        puts("ERR_FUNDS"); break;
                    }
                    balance -= amount;
                }
                puts("OK"); break;
            case 3:
                printf("Balance: %lld.%02lld\n",
                    balance / 100, balance % 100);
                break;
            case 4:
                puts("BYE"); return 0;
            default:
                puts("ERR_CMD"); break;
        }
    }
}
```

[跳至主要內容](#)

```
gcc -std=c11 -Wall -Wextra -Wpedantic c15_ledger.c -o c15_ledger
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

15 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

C15-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 21 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1 104256 2 13379 3 4	OK OK Balance: 908.77 BYE
21 bytes · 最後有 LF 換行	26 bytes · 包含程式實際輸出的換行與空白

C15-T02 初始餘額為0

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
3 4	Balance: 0.00 BYE
4 bytes · 最後有 LF 換行	18 bytes · 包含程式實際輸出的換行與空白

C15-T03 驗證 ERR_FUNDS

exit 0 ✓

原資源包 · 8 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
跳至主要內容 4	ERR_FUNDS Balance: 0.00 BYE

8 bytes · 最後有 LF 換行

28 bytes · 包含程式實際輸出的換行與空白

—

C15-T04 一分要印成0.01元

exit 0 ✓

原資源包 · 8 input bytes · 完整比對

STDIN · 完整輸入

1 1
3
4

8 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

OK
Balance: 0.01
BYE

21 bytes · 包含程式實際輸出的換行與空白

—

C15-T05 驗證 ERR_AMOUNT

exit 0 ✓

原資源包 · 9 input bytes · 完整比對

STDIN · 完整輸入

1 -1
3
4

9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_AMOUNT
Balance: 0.00
BYE

29 bytes · 包含程式實際輸出的換行與空白

—

C15-T06 驗證 ERR_CMD

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入

8
4

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_CMD
BYE

12 bytes · 包含程式實際輸出的換行與空白

—

C15-T07 收入達上限後再收入 · 拒絕且餘額不變

exit 0 ✓

原資源包 · 24 input bytes · 完整比對

跳至主要內容

1 1000000000000
1 1

STDOUT · 完整預期輸出

OK
ERR_LIMIT

3
4

24 bytes · 最後有 LF 換行

Balance: 1000000000.00
BYE

41 bytes · 包含程式實際輸出的換行與空白

C15-T08 清除帳本後餘額歸0

原資源包 · 12 input bytes · 完整比對

exit 0 ✓

STDIN · 完整輸入

1 123
0
3
4

12 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

OK
CLEARED
Balance: 0.00
BYE

29 bytes · 包含程式實際輸出的換行與空白

C15-T09 驗證 ERR_INPUT

原資源包 · 1 input bytes · 完整比對

exit 1 ✓

STDIN · 完整輸入

1

1 bytes · 最後沒有換行，檔案直接結束

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

C15-T10 空輸入檔：立即 EOF

原資源包 · 0 input bytes · 完整比對

exit 0 ✓

STDIN · 完整輸入

(空輸入檔，直接 EOF)

0 bytes · 0 bytes · 不是空白行

STDOUT · 完整預期輸出

(空輸出)

0 bytes · 包含程式實際輸出的換行與空白

C15-T11 非數字 token：scanf 讀取失敗

L 新增測試 · 2 input bytes · 完整比對

exit 1 ✓

跳至主要內容

STDIN · 完整輸入

STDOUT · 完整預期輸出

x
2 bytes · 最後有 LF 換行

ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

- C15-T12 支出剛好用完餘額

exit 0 ✓
- 本 HTML 新增測試 · 16 input bytes · 完整比對

STDIN · 完整輸入
1 100 2 100 3 4
16 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
OK OK Balance: 0.00 BYE
24 bytes · 包含程式實際輸出的換行與空白

- C15-T13 零金額被拒絕，但可繼續接收指令

exit 0 ✓
- 本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入
1 0 4
6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_AMOUNT BYE
15 bytes · 包含程式實際輸出的換行與空白

- C15-T14 單筆金額超限

exit 0 ✓
- 本 HTML 新增測試 · 18 input bytes · 完整比對

STDIN · 完整輸入
1 1000000000001 4
18 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_AMOUNT BYE
15 bytes · 包含程式實際輸出的換行與空白

- 跳至主要內容: 出後餘額保持不變

exit 0 ✓
- L 新增測試 · 16 input bytes · 完整比對

STDIN · 完整輸入

1 100
2 101
3
4

16 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

OK
ERR_FUNDS
Balance: 1.00
BYE

31 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 原教材採 float 餘額；本版新增「整數分」表示及交易防護。
- 本案例僅是狀態與控制流程練習，不是可直接部署的金融系統。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

班級成績統計器

while

continue

統計

有效筆數

對應整合版：第 88–92 頁 · 1 份程式 · 11 組測試

來源對照：整合 Ch5 分支、Ch6 迴圈、Ch7 型別與轉換

題目、規則與程式沿用既有資源包；本 HTML 另補變數表、追蹤、正確性說明與測試用途標籤。

01 完整題目

先確認規格，再寫程式

輸入預計蒐集的合法成績筆數 n ，逐筆處理成績。超出 0 ~ 100 的值略過並要求下一筆，蒐集滿 n 筆後輸出平均、最高、最低與及格人數。

輸入規格

第一個整數 n 為 1 ~ 1000；後續為成績整數。最多處理 10000 筆成績輸入。

輸出規格

無效成績輸出 IGNORED；蒐集完成輸出 Count、Mean（2 位）、Min、Max、Pass；輸入中斷 ERR_INPUT；超過嘗試上限 ERR_LIMIT。

規則與限制

- 及格定義 $\text{score} \geq 60$ ；無效成績不計入 n 、總和或極值。
- 要累積「 n 筆有效值」，不是只讀 n 次。
- n 非法輸出 ERR_RANGE；最多 10000 次嘗試防止無限制重試。

輸入 / 輸出範例

STDIN · 範例輸入

```
4
90 101 59 60 100
```

19 bytes · 最後有 LF 換行

STDOUT · 範例輸出

```
IGNORED
Count: 4
Mean: 77.25
Min: 59
Max: 100
Pass: 3
```

54 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

輸入範例已另存為檔案。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1
- 以 $count < n$ 作為條件，count 只在接受合法資料後增加。
- 2
- 四項更新 sum、min、max、pass 都只能處理合法成績。
- 3
- 已保證 $n \geq 1$ ，完成後 $count > 0$ ，可安全計算平均。

重要變數與狀態

名稱	型別 / 形式	用途
n / count / tries	int	需求有效筆數 / 已接受 / 嘗試次數
score / min / max / pass	int	本筆分數 / 極值 / 及格人數
sum	long long	合法分數總和

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

輸入	tries	count	sum	min / max	pass
90	1	1	90	90 / 90	1
101	2	1	90	90 / 90	1
59	3	2	149	59 / 90	1
60	4	3	209	59 / 90	2
100	5	4	309	59 / 100	3

為什麼這個解法成立？

count 只在接受合法值後增加。sum、極值與 pass 同時只處理合法資料，因此完成時恰好統計 n 筆；tries 額外限制反覆輸入無效，跳至主要內容 ≥ 1 ，平均除法有非零分母。

t 為嘗試讀取筆數：時間 $O(t)$ ， $t \leq 10000$ ；額外空間 $O(1)$ 。

03 完整程式碼

每一份都可獨立編譯

c16_score_stats.c

完整程式 · 30 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int n, score, count = 0, tries = 0, pass = 0;
    int min = 100, max = 0;
    long long sum = 0;
    if (scanf("%d", &n) != 1) {
        puts("ERR_INPUT"); return 1;
    }
    if (n < 1 || n > 1000) {
        puts("ERR_RANGE"); return 1;
    }
    while (count < n) {
        if (tries == 1000) { puts("ERR_LIMIT"); return 1; }
        if (scanf("%d", &score) != 1) {
            puts("ERR_INPUT"); return 1;
        }
        tries++;
        if (score < 0 || score > 100) {
            puts("IGNORED"); continue;
        }
        sum += score;
        count++;
        if (score < min) min = score;
        if (score > max) max = score;
        if (score >= 60) pass++;
    }
    printf("Count: %d\nMean: %.2f\n", count, (double)sum / count);
    printf("Min: %d\nMax: %d\nPass: %d\n", min, max, pass);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行)：

```
gcc -std=c11 -Wall -Wextra -Wpedantic c16_score_stats.c -o c16_score_stats
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

0 跳至主要內容 ↓

11 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

C16-T01 題目示範輸入 / 輸出

exit 0 ✓

原資源包 · 19 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
<pre>4 90 101 59 60 100</pre>	<pre>IGNORED Count: 4 Mean: 77.25 Min: 59 Max: 100 Pass: 3</pre>
19 bytes · 最後有 LF 換行	54 bytes · 包含程式實際輸出的換行與空白

C16-T02 單筆0分，最低最高均為0

exit 0 ✓

原資源包 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
<pre>1 0</pre>	<pre>Count: 1 Mean: 0.00 Min: 0 Max: 0 Pass: 0</pre>
4 bytes · 最後有 LF 換行	42 bytes · 包含程式實際輸出的換行與空白

C16-T03 單筆100分，最低最高均為100

exit 0 ✓

原資源包 · 6 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
<pre>1 100</pre>	<pre>Count: 1 Mean: 100.00 Min: 100 Max: 100 Pass: 1</pre>
6 bytes · 最後有 LF 換行	48 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

C16-T04 59/60分的及格界線與非整數平均

exit 0 ✓

原資源包 · 8 input bytes · 完整比對

STDOUT · 完整預期輸出

```
Count: 2
Mean: 59.50
Min: 59
Max: 60
Pass: 1
```

45 bytes · 包含程式實際輸出的換行與空白

```
exit 1 ✓
```

原資源包 · 4 input bytes · 完整比對

STDOUT · 完整預期輸出

ERR INPUT

10 bytes · 包含程式實際輸出的換行與空白

```
exit 1 ✓
```

原資源包 · 2 input bytes · 完整比對

STDOUT · 完整預期輸出

ERR RANGE

10 bytes · 包含程式實際輸出的換行與空白

```
exit 1 ✓
```

原資源包上限測試 · 40,002 input bytes · 完整比對

STDOUT · 完整預期輸出

IGNORED
IGNORED
IGNORED
IGNORED
IGNORED
IGNORED

40,002 bytes · 最後沒有換行，檔案直接結束

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

[跳至主要內容](#)

[illegible]

ERR_LIMIT

[跳至主要內容](#)

80,010 bytes · 包含程式實際輸出的換行與空白

C16-T08 非數字 token : scanf 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

C16-T09 需求筆數超出上限

exit 1 ✓

本 HTML 新增測試 · 5 input bytes · 完整比對

STDIN · 完整輸入

1001

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

C16-T10 無效值不汙染極值與平均

exit 0 ✓

本 HTML 新增測試 · 15 input bytes · 完整比對

STDIN · 完整輸入

2 -1 0 101 100

15 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

IGNORED
IGNORED
Count: 2
Mean: 50.00
Min: 0
Max: 100
Pass: 1

61 bytes · 包含程式實際輸出的換行與空白

C16-T11 需求筆數最大值 · 全部剛好及格

exit 0 ✓

跳至主要內容 L 新增測試 · 3,005 input bytes · 完整比對

STDIN · 完整輸入

STDOUT · 完整預期輸出

<pre>1000 60</pre>	<pre>Count: 1000 Mean: 60.00 Min: 60 Max: 60 Pass: 1000</pre>
3,005 bytes · 最後沒有換行，檔案直接結束	51 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- min=100、max=0 依賴本題已知成績範圍；一般資料可用第一筆合法值初始化。
- for(i=0;i<n;i++) 若遇無效成績仍 i++，會提早停止。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

比較、連鎖不等式與否定的解析

比較

區間

指派

對應整合版：第 6、9、28、93 頁 · 1 份程式 · 6 組測試

來源對照：整合版 p.6,9,28,93 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

讀取一個整數 x ，分別觀察正確區間判斷、錯誤連鎖比較的實際解析、否定括號，以及 $==$ 與 $=$ 的差異。最後印出 x 的副本被指派成 0 後的值。

輸入規格

一個整數 x ， $-1000 \sim 1000$ 。

輸出規格

依序輸出 Compare、Nonzero、WrongRange、CorrectRange、WrongNot、CorrectNot、EqualZero、AssignValue、AfterAssign。

規則與限制

- WrongRange 以 $first=(0 \leq x)$ ，再計算 $first < 10$ ，明示原式 $0 \leq x < 10$ 的實際順序。
- CorrectRange 是 $0 \leq x \ \&\& \ x < 10$ ；本題半開區間與第93頁的 $0 \sim 100$ 閉區間問題不同。
- 範圍或讀取錯誤印 ERR_RANGE 或 ERR_INPUT，結束碼為1。
- Compare 輸出 $3 < 5$ ；Nonzero 輸出 $!(-7)$ ，兩者皆以0或1表示。
- WrongNot 為 $!(x) == 2$ ；CorrectNot 為 $!(x) == 2$ 。先令 $copy = x$ ，EqualZero輸出 $copy == 0$ ，再執行 $copy = 0$ ；AssignValue與AfterAssign分別輸出指派式的值與copy最後的值。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

跳至主要內容

3 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Compare: 1
Nonzero: 1
WrongRange: 1
CorrectRange: 0

```
WrongNot: 0
CorrectNot: 1
EqualZero: 0
AssignValue: 0
AfterAssign: 0
```

121 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 將連鎖比較拆成兩次比較。
- 2 正確區間使用 &&。
- 3 先分清否定的是 x 還是整個比較。
- 4 指派式的值為被指定的值。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

觀察項目	x=50 的結果
0<=50	1
1<10	1 (錯誤區間也成立)
0<=50 && 50<10	0
copy=0	修改 copy，表達式也得到0

為什麼這個解法成立？

拆| 跳至主要內容 | 連鎖比較的語意，並直接與正確區間式比對。

本題資料規模已限制；重點為所示控制流程與型別行為。

03 完整程式碼

每一份都可獨立編譯

s01_comparison.c

補齊後的完整程式 · 20 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int x;
    if (scanf("%d", &x) != 1) { puts("ERR_INPUT"); return 1; }
    if (x < -1000 || x > 1000) { puts("ERR_RANGE"); return 1; }
    int first = (0 <= x);
    int negated = !x;
    int copy = x;
    int equal_zero = (copy == 0);
    int assigned = (copy = 0);
    printf("Compare: %d\n", 3 < 5);
    printf("Nonzero: %d\n", !!( -7 ));
    printf("WrongRange: %d\n", first < 10);
    printf("CorrectRange: %d\n", 0 <= x && x < 10);
    printf("WrongNot: %d\n", negated == 2);
    printf("CorrectNot: %d\n", !(x == 2));
    printf("EqualZero: %d\n", equal_zero);
    printf("AssignValue: %d\nAfterAssign: %d\n", assigned, copy);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s01_comparison.c -o s01_comparison
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

6 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— s01-T01 x=50：錯誤區間仍成立

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

跳至主要內容

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Compare: 1
Nonzero: 1
WrongRange: 1
CorrectRange: 0

```
WrongNot: 0
CorrectNot: 1
EqualZero: 0
AssignValue: 0
AfterAssign: 0
```

121 bytes · 包含程式實際輸出的換行與空白

S01-T02 包含的下界

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

```
Compare: 1
Nonzero: 1
WrongRange: 1
CorrectRange: 1
WrongNot: 0
CorrectNot: 1
EqualZero: 1
AssignValue: 0
AfterAssign: 0
```

121 bytes · 包含程式實際輸出的換行與空白

S01-T03 不包含的上界

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

10

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

```
Compare: 1
Nonzero: 1
WrongRange: 1
CorrectRange: 0
WrongNot: 0
CorrectNot: 1
EqualZero: 0
AssignValue: 0
AfterAssign: 0
```

121 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

S01-T04 否定 x==2

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

2

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Compare: 1
Nonzero: 1
WrongRange: 1
CorrectRange: 1
WrongNot: 0
CorrectNot: 0
EqualZero: 0
AssignValue: 0
AfterAssign: 0

121 bytes · 包含程式實際輸出的換行與空白

S01-T05 負數不在區間

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Compare: 1
Nonzero: 1
WrongRange: 1
CorrectRange: 0
WrongNot: 0
CorrectNot: 1
EqualZero: 0
AssignValue: 0
AfterAssign: 0

121 bytes · 包含程式實際輸出的換行與空白

S01-T06 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

10 bytes · 最後有 LF 換行

跳至主要內容

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 程式刻意展示錯誤條件，但用暫存變數呈現解析順序，避免把錯誤寫法當成推薦用法。
- 非零條件為真，不代表每個真值本來都是1。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

短路求值與被略過的自增

短路

&&

||

對應整合版：第 7、28 頁 · 1 份程式 · 4 組測試

來源對照：整合版 p.7,28 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

輸入 total、den、i、j。先用短路條件保護除法，再分別觀察 `i>0 && ++j>0` 及 `i>0 || ++j>0`。AND 與 OR 示範都從同一個原始 j 開始。

輸入規格

四個整數 total den i j，各為 -10000 ~ 10000。

輸出規格

印 Guard、And、AfterAnd、Or、AfterOr。

規則與限制

- Guard 使用 `den!=0 && total/den>2`，這裡是整數除法。
- 在兩個自增示範之間將 j 還原，不能讓第一個示範影響第二個。
- 範圍有限，避免除法與自增超出型別範圍。
- And、Or是兩個邏輯式的0/1結果；AfterAnd、AfterOr是各自運算後的j，不是邏輯值。輸出順序固定為Guard、And、AfterAnd、Or、AfterOr。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

100 0 0 5

10 bytes · 最後有 LF 換行

跳至主要內容

STDOUT · 範例輸出

Guard: 0
And: 0
AfterAnd: 5
Or: 1
AfterOr: 6

45 bytes · 包含程式實際輸出的換行與空白

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 先確認分母再做除法。
- 2 AND 左側假，右側不做。
- 3 OR 左側真，右側不做。
- 4 分別列出自增後狀態。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

路徑	i=0、j=5
AND 左側	假；++j不執行
AfterAnd	5
OR 左側	假；須評估++j
AfterOr	6

為什麼這個解法成立？

是否執行右側取決於左側是否足以決定整個邏輯式。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

跳至主要內容

.t.c

行・長行可橫向捲動

```
#include <stdio.h>
int main(void) {
```

```
int total, den, i, j;
if (scanf("%d %d %d %d", &total, &den, &i, &j) != 4) {
    puts("ERR_INPUT"); return 1;
}
if (total < -10000 || total > 10000 || den < -10000 || den > 10000 ||
    i < -10000 || i > 10000 || j < -10000 || j > 10000) {
    puts("ERR_RANGE"); return 1;
}
int original = j;
int guard = den != 0 && total / den > 2;
int and_result = i > 0 && ++j > 0;
printf("Guard: %d\nAnd: %d\nAfterAnd: %d\n", guard, and_result, j);
j = original;
int or_result = i > 0 || ++j > 0;
printf("Or: %d\nAfterOr: %d\n", or_result, j);
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s02_short_circuit.c -o s02_short_circuit
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

4 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

– S02-T01 零分母與 AND 短路

exit 0 ✓

本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
100 0 0 5	Guard: 0 And: 0 AfterAnd: 5 Or: 1 AfterOr: 6
10 bytes · 最後有 LF 換行	45 bytes · 包含程式實際輸出的換行與空白

– 跳至主要內容 路

exit 0 ✓

本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入

100 10 1 5

11 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Guard: 1
And: 1
AfterAnd: 6
Or: 1
AfterOr: 5

45 bytes · 包含程式實際輸出的換行與空白

S02-T03 自增成0仍為假

exit 0 ✓

本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入

1 2 0 -1

9 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Guard: 0
And: 0
AfterAnd: -1
Or: 0
AfterOr: 0

46 bytes · 包含程式實際輸出的換行與空白

S02-T04 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 跳至主要內容 發生的更新藏在可能被短路的分支裡。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

大括號與 else 配對

if

else

大括號

對應整合版：第 8–9 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.8,9 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

輸入 outer、inner，分別模擬「else 配對內層 if」與「else 配對外層 if」兩種結構，輸出各自是否執行 ACTION、FALLBACK 或沒有動作。

輸入規格

兩個整數 outer inner，只能是0或1。

輸出規格

印 InnerElse: 動作及 OuterElse: 動作。

規則與限制

- 用大括號將兩段的語法結構明示，行為對應第8頁左右兩種配對。
- 每段都必須輸出一個結果；NONE 表示原片段不執行任何動作。
- 內層配對：outer=0時NONE；outer=1且inner=1時ACTION；outer=1且inner=0時FALLBACK。
- 外層配對：outer=0時FALLBACK；outer=1且inner=1時ACTION；outer=1且inner=0時NONE。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

1 0

4 bytes · 最後有 LF 換行

STDOUT · 範例輸出

InnerElse: FALLBACK
OuterElse: NONE

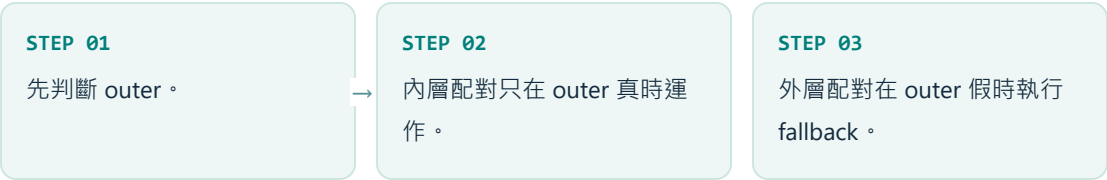
36 bytes · 包含程式實際輸出的換行與空白

輸，跳至主要內容 ²。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 先判斷 outer。
- 2 內層配對只在 outer 真時運作。
- 3 外層配對在 outer 假時執行 fallback。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

outer / inner	內層else	外層else
1 / 0	FALLBACK	NONE
0 / 1	NONE	FALLBACK

為什麼這個解法成立？

兩段具有不同語法結構，因此不能因縮排看起來相近就認為等價。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s03_dangling_else.c

補齊後的完整程式 · 20 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int outer, inner;
    if (scanf("%d %d", &outer, &inner) != 2) { puts("ERR_INPUT"); return 1; }
    if (inner != 0 && outer != 1) || (inner != 0 && inner != 1)) {
        跳至主要內容 跳至範圍"); return 1;
    }
    const char *a = "NONE", *b = "NONE";
    if (outer) {
        if (inner) a = "ACTION";
    }
}
```



```
        else a = "FALLBACK";
    }
    if (outer) {
        if (inner) b = "ACTION";
    } else {
        b = "FALLBACK";
    }
    printf("InnerElse: %s\nOuterElse: %s\n", a, b);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s03_dangling_else.c -o s03_dangling_else
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— S03-T01 外真、內假

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
1 0	InnerElse: FALLBACK OuterElse: NONE
4 bytes · 最後有 LF 換行	36 bytes · 包含程式實際輸出的換行與空白

— S03-T02 外假、內層不執行

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0 1	InnerElse: NONE OuterElse: FALLBACK
跳至主要內容 · 換行	36 bytes · 包含程式實際輸出的換行與空白

— S03-T03 兩條件皆真

exit 0 ✓

STDIN · 完整輸入

1 1

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

InnerElse: ACTION
OuterElse: ACTION

36 bytes · 包含程式實際輸出的換行與空白

S03-T04 兩條件皆假

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

0 0

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

InnerElse: NONE
OuterElse: FALLBACK

36 bytes · 包含程式實際輸出的換行與空白

S03-T05 非法旗標

exit 1 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

2 0

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- const char * 在此僅用來保存輸出標籤；屬本 HTML 為補齊輸出新增的簡單用法。
- 更改縮排不會改變 else 配對。

跳至主要內容

數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制
元六四制八六制試規則。

bool、條件運算子與獨立 if

bool ?: else if

對應整合版：第 9、17、28 頁 · 1 份程式 · 4 組測試

來源對照：整合版 p.9,17,28 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01 完整題目

先確認規格，再寫程式

輸入 a、b 與 flag，將 flag 轉成 bool，找出 a、b 最大值，觀察 ?: 只自增被選中的變數，並比較兩個獨立 if 與 if/else if 命中的次數。

輸入規格

三個整數 a b flag，各在 -10000 ~ 10000。

輸出規格

印 Ready、Max、Left、Right、Independent、Chained。

規則與限制

- flag=0代表false；其他值轉成true。
- left與right從0開始；ready為真時只增加left。
- 獨立判斷為 $a \geq 0$ 與 $a \geq 10$ ；串接版本只計第一個成立分支。
- Ready以0/1輸出轉成bool後的flag；Max為a、b最大值。Ready為假時只將right由0增為1；Left與Right輸出各自最後的值。
- Independent分別計數 $a \geq 0$ 、 $a \geq 10$ 兩個成立條件；Chained依if($a \geq 0$)、else if($a \geq 10$)的順序只計第一個成立者。本題刻意比較兩種結構。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

10 2 5

跳至主要內容

7 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Ready: 1

Max: 10

Left: 1

Right: 0

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 將旗標轉成 bool。
- 2 用 ?: 選值及單一支。
- 3 獨立 if 可各自成立。
- 4 else if 只取第一個成立分支。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

步驟	輸入10 2 5
ready	1
maximum	10
?:	left=1 · right=0
命中數	Independent=2 · Chained=1

為什麼這個解法成立？

bool非零正規化、條件分支單選與if串接互斥，可分別由列印出的狀態核對。

；重點為所示控制流程與型別行為。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

s04_bool_ternary.c

補齊後的完整程式 · 21 行 · 長行可橫向捲動

```
#include <stdio.h>
#include <stdbool.h>
int main(void) {
    int a, b, flag;
    if (scanf("%d %d %d", &a, &b, &flag) != 3) { puts("ERR_INPUT"); return 1; }
    if (a < -10000 || a > 10000 || b < -10000 || b > 10000 ||
        flag < -10000 || flag > 10000) { puts("ERR_RANGE"); return 1; }
    bool ready = flag;
    int maximum = a > b ? a : b;
    int left = 0, right = 0;
    int chosen = ready ? ++left : ++right;
    (void)chosen;
    int independent = 0, chained = 0;
    if (a >= 0) independent++;
    if (a >= 10) independent++;
    if (a >= 0) chained++;
    else if (a >= 10) chained++;
    printf("Ready: %d\nMax: %d\nLeft: %d\nRight: %d\n", ready, maximum, left, right);
    printf("Independent: %d\nChained: %d\n", independent, chained);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s04_bool_ternary.c -o s04_bool_ternary
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

4 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— S04-T01 非零正規化與兩個if命中

exit 0 ✓

本 HTML 新增測試 · 7 input bytes · 完整比對

跳至主要內容

10 2 5

STDOUT · 完整預期輸出

Ready: 1
Max: 10
Left: 1

7 bytes · 最後有 LF 換行

Right: 0
Independent: 2
Chained: 1

60 bytes · 包含程式實際輸出的換行與空白

S04-T02 false選右分支

exit 0 ✓

本 HTML 新增測試 · 8 input bytes · 完整比對

STDIN · 完整輸入

-3 -1 0

8 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Ready: 0
Max: -1
Left: 0
Right: 1
Independent: 0
Chained: 0

60 bytes · 包含程式實際輸出的換行與空白

S04-T03 相等值與負的非零旗標

exit 0 ✓

本 HTML 新增測試 · 7 input bytes · 完整比對

STDIN · 完整輸入

2 2 -7

7 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Ready: 1
Max: 2
Left: 1
Right: 0
Independent: 1
Chained: 1

59 bytes · 包含程式實際輸出的換行與空白

S04-T04 a=0門檻

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入

跳至主要內容

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Ready: 0
Max: 1
Left: 0
Right: 1

Independent: 1
Chained: 1

59 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 此處自增只是觀察求值行為；一般流程仍宜保留清楚的if區塊。
- 串接範例的第二個條件刻意無法命中，用來對比獨立if。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

switch 分派、共用標籤與 fall-through

switch

break

fall-through

對應整合版：第 18–19 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.18,19 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

輸入 command 與 grade。第一段依command分派命令；第二段刻意不在各grade之間break，重現教材從入口一路向下執行的字串串接。

輸入規格

兩個整數 command grade，各在-10 ~ 10。

輸出規格

第一行為命令文字；第二行為從grade入口開始串接的文字。

規則與限制

- command=1輸出START、2輸出STOP、3或4輸出REPORT，其餘輸出ERR_CMD。
- grade=4/3/2/1/0分別從Excellent/Good/Average/Poor/Failing開始，最後流入Illegal grade。
- 這是刻意的fall-through示範，並非一般成績分類程式。
- grade=4時第二行完整字串為ExcellentGoodAveragePoorFailingIllegal grade；其他合法grade值由對應字串起印，之間不另加空白。grade不為0 ~ 4時只印Illegal grade。
- ERR_CMD是command未匹配的正常輸出，仍需印grade結果；只要兩個輸入均在-10 ~ 10，程式結束碼仍為0。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

3 3

跳至主要內容

4 bytes · 最後有 LF 換行

STDOUT · 範例輸出

REPORT

GoodAveragePoorFailingIllegal grade

43 bytes · 包含程式實際輸出的換行與空白

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1
- command先選入口。
- 2
- break終止單次switch。
- 3
- 共用標籤共用實作。
- 4
- grade段刻意向下流動。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

路徑	grade=3
入口	Good
接續	Average → Poor → Failing
最後	Illegal grade

為什麼這個解法成立？

沒有break時，後續case與default不會重新測試grade，而是依程式順序執行。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s05_switch_paths.c

補齊後的完整程式 · 24 行 · 長行可橫向捲動

跳至主要內容

: 1>

int main(void) {
 int command, grade;
 if (scanf("%d %d", &command, &grade) != 2) { puts("ERR_INPUT"); return 1; }

```
if (command < -10 || command > 10 || grade < -10 || grade > 10) {
    puts("ERR_RANGE"); return 1;
}
switch (command) {
case 1: puts("START"); break;
case 2: puts("STOP"); break;
case 3: case 4: puts("REPORT"); break;
default: puts("ERR_CMD"); break;
}
switch (grade) {
case 4: printf("Excellent"); /* fall through */
case 3: printf("Good"); /* fall through */
case 2: printf("Average"); /* fall through */
case 1: printf("Poor"); /* fall through */
case 0: printf("Failing"); /* fall through */
default: printf("Illegal grade"); break;
}
putchar('\n');
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s05_switch_paths.c -o s05_switch_paths
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S05-T01 共用標籤與grade=3串接

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
3 3	REPORT
4 bytes · 最後有 LF 換行	GoodAveragePoorFailingIllegal grade
	43 bytes · 包含程式實際輸出的換行與空白

—

跳至主要內容 · 標籤開始

exit 0 ✓

L 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

4 4

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

REPORT
ExcellentGoodAveragePoorFailingIllegal grade

52 bytes · 包含程式實際輸出的換行與空白

- S05-T03

grade=0仍流入default

exit 0 ✓
- 本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

1 0

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

START
FailingIllegal grade

27 bytes · 包含程式實際輸出的換行與空白

- S05-T04

沒有符合標籤

exit 0 ✓
- 本 HTML 新增測試 · 5 input bytes · 完整比對

STDIN · 完整輸入

2 -1

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

STOP
Illegal grade

19 bytes · 包含程式實際輸出的換行與空白

- S05-T05

default命令但程式仍繼續

exit 0 ✓
- 本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

8 2

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_CMD
AveragePoorFailingIllegal grade

40 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- ERR_CMD在本示範只代表未知命令，程式仍完成grade示範，結束碼為0。
- 不要把標籤當成自動隔離的分支區塊。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

邏輯 AND 與低8位元運算

位元

unsigned

&&

對應整合版：第 27 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.27 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01 完整題目

先確認規格，再寫程式

輸入0~255的兩個整數a、b，比較邏輯AND和位元AND，並列出OR、XOR、a的低8位反相及左右移2位結果。

輸入規格

兩個整數a b，各在0~255。

輸出規格

印 LogicalAND、AND、OR、XOR、NOT_low8、Left2、Right2。

規則與限制

- 先驗證範圍，再轉為unsigned int進行位元操作。
- ~a反轉整個unsigned int；本題以&0xFF只保留低8位。
- Left2輸出整個unsigned結果，不截成8位。
- LogicalAND為(a&&b)的0/1結果；AND、OR、XOR分別為a&b、a|b、a^b。NOT_low8為(~a)&0xFF，Left2為a<<2，Right2為a>>2；全部以十進位顯示。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

60 13

6 bytes · 最後有 LF 換行

跳至主要內容

STDOUT · 範例輸出

LogicalAND: 1
AND: 12
OR: 61
XOR: 49
NOT_low8: 195
Left2: 240
Right2: 15

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 把a、b視為二進位數。
- 2 &&先視為真假。
- 3 &、|、^逐bit運算。
- 4 反相後套低8位遮罩。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

運算	60與13結果
60 && 13	1
60 & 13	12
60 13	61
60 ^ 13	49
(~60U)&255U	195

為什麼這個解法成立？

邏輯運算結果僅為0或1；位元運算結果則保存各bit位置的計算結果。

本題資料規模已限制；重點為所示控制流程與型別行為。

跳至主要內容

03

完整程式碼

每一份都可獨立編譯

s06_bitwise.c

補齊後的完整程式 · 12 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int ia, ib;
    if (scanf("%d %d", &ia, &ib) != 2) { puts("ERR_INPUT"); return 1; }
    if (ia < 0 || ia > 255 || ib < 0 || ib > 255) { puts("ERR_RANGE"); return 1; }
    unsigned int a = (unsigned int)ia, b = (unsigned int)ib;
    printf("LogicalAND: %d\n", a && b);
    printf("AND: %u\nOR: %u\nXOR: %u\n", a & b, a | b, a ^ b);
    printf("NOT_low8: %u\n", (~a) & 0xFFU);
    printf("Left2: %u\nRight2: %u\n", a << 2, a >> 2);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s06_bitwise.c -o s06_bitwise
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— S06-T01 教材60與13

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入 60 13 6 bytes · 最後有 LF 換行 跳至主要內容	STDOUT · 完整預期輸出 LogicalAND: 1 AND: 12 OR: 61 XOR: 49 NOT_low8: 195 Left2: 240 Right2: 15 73 bytes · 包含程式實際輸出的換行與空白
---	---

S06-T02 含零與全1低8位

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入

0 255

6 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

LogicalAND: 0
AND: 0
OR: 255
XOR: 255
NOT_low8: 255
Left2: 0
Right2: 0

71 bytes · 包含程式實際輸出的換行與空白

S06-T03 最大合法值

exit 0 ✓

本 HTML 新增測試 · 8 input bytes · 完整比對

STDIN · 完整輸入

255 255

8 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

LogicalAND: 1
AND: 255
OR: 255
XOR: 0
NOT_low8: 0
Left2: 1020
Right2: 63

73 bytes · 包含程式實際輸出的換行與空白

S06-T04 真假相同但共同bit為0

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

1 2

4 bytes · 最後有 LF 換行

跳至主要內容

STDOUT · 完整預期輸出

LogicalAND: 1
AND: 0
OR: 3
XOR: 3
NOT_low8: 254
Left2: 4
Right2: 0

67 bytes · 包含程式實際輸出的換行與空白

— S06-T05 負值拒絕

exit 1 ✓

本 HTML 新增測試 · 5 input bytes · 完整比對

STDIN · 完整輸入

-1 0

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- 圖中的8格不代表unsigned int只有8位。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

計數端點、逗號運算子與多變數更新

for

comma

作用域

對應整合版：第 37 頁 · 1 份程式 · 4 組測試

來源對照：整合版 p.37 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

輸入 n ，列印 $0 \sim n-1$ 與 $1 \sim n$ 兩種計數，輸出外部宣告的迴圈變數終值，再觀察教材的逗號運算子與三變數 for 更新。

輸入規格

一個整數 n ， $0 \sim 9$ 。

輸出規格

印 Zero、One、After 及 Comma；接著逐列印 a,b,c 三個狀態。

- 規則與限制
- $n=0$ 時前兩個序列皆為空，三變數迴圈也不執行。
 - 逗號運算使用 $j=10$ ，計算 $(j++, (void)(j+100), 999+j)$ ；void 明示丟棄中間值。
 - 三變數初值 $a=1, b=10, c=1$ ，每回合 $a+1$ 、 $b-1$ 、 $c \times 2$ 。
 - Zero: 與 One: 後的每個數字前面各有一個空白；空序列只印標籤及換行。After 印第一個 for 結束時 i 的值，Comma 印逗號式最後得到的 i 與 j ，中間一個空白。
 - 最後三變數迴圈的條件為 $a \leq n$ ，每一輪先印出 a b c (三者以空白分隔) 再更新，總共 n 列；不輸出欄名。
 - 共同錯誤處理：讀取數值失敗印 ERR_INPUT；數值可讀取但超出本題輸入範圍印 ERR_RANGE，皆以結束碼 1 停止。正常完成以 0 結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

3

跳至主要內容 行

STDOUT · 範例輸出

Zero: 0 1 2
One: 1 2 3
After: 3
Comma: 1010 11
1 10 1

2026/10/4 下午4:32

C 程式設計 | 完整案例實作手冊

2 9 2

3 8 4

66 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 列出兩種端點區間。
- 2 外部i可在迴圈後讀取。
- 3 逗號先左後右，值取最後。
- 4 每回合同步更新a,b,c。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

步驟	j與i
j初值	10
j++	j=11
j+100	111被丟棄
999+j	i=1010

為什麼這個解法成立？

0 ~ n-1與1 ~ n皆有n個元素；三個更新式由逗號順序執行，印出的是每回合更新前的狀態。

本題資料規模已限制；重點為所示控制流程與型別行為。

跳至主要內容

s07_count_comma.c
補齊後的完整程式 · 18 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int n, i;
    if (scanf("%d", &n) != 1) { puts("ERR_INPUT"); return 1; }
    if (n < 0 || n > 9) { puts("ERR_RANGE"); return 1; }
    printf("Zero:");
    for (i = 0; i < n; i++) printf(" %d", i);
    printf("\nOne:");
    for (int k = 1; k <= n; k++) printf(" %d", k);
    printf("\nAfter: %d\n", i);
    int j = 10;
    i = (j++, (void)(j + 100), 999 + j);
    printf("Comma: %d %d\n", i, j);
    for (int a = 1, b = 10, c = 1; a <= n; a++, b--, c *= 2) {
        printf("%d %d %d\n", a, b, c);
    }
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s07_count_comma.c -o s07_count_comma
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

- S07-T01 n=3兩種區間
- exit 0 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
3 跳至主要內容 2 bytes · 最後有 LF 換行	Zero: 0 1 2 One: 1 2 3 After: 3 Comma: 1010 11 1 10 1

2 9 2

3 8 4

66 bytes · 包含程式實際輸出的換行與空白

- S07-T02

零次迴圈

exit 0 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Zero:

One:

After: 0

Comma: 1010 11

35 bytes · 包含程式實際輸出的換行與空白

- S07-T03

最大合法次數

exit 0 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

9

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Zero: 0 1 2 3 4 5 6 7 8

One: 1 2 3 4 5 6 7 8 9

After: 9

Comma: 1010 11

1 10 1

2 9 2

3 8 4

4 7 8

5 6 16

6 5 32

7 4 64

8 3 128

9 2 256

133 bytes · 包含程式實際輸出的換行與空白

- S07-T04

超出上限

exit 1 ✓
- 本 HTML 新增測試 · 3 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入

STDOUT · 完整預期輸出

10	ERR_RANGE
3 bytes · 最後有 LF 換行	10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- printf引數間的逗號不是逗號運算子，不保證引數由左到右求值。
- (void)是本HTML新增的明示丟棄，保持中間計算對最終結果的影響不變。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

不用乘法的奇數累加平方表

for

平方差

對應整合版：第 33、34、37 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.33,34,37 (原題 / 語意依投影片及備忘錄) ；另參照原 Chapter 6 Loops.pptx 第45–46頁的 square3.c 。

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

沿用C06的平方表輸入輸出，但不可用乘法計算平方。利用相鄰平方差為3、5、7...，由1開始累加下一個奇數。此題補齊原Ch6 square3.c的另一種方法。

輸入規格

一個整數n，1 ~ 20。

輸出規格

每列輸出i及i²，中間一個空白，共n列。

- 規則與限制
- 初值i=1、square=1、odd=3。
 - 印完目前平方後，square+=odd，再讓odd+=2。
 - 輸出必須與C06的while/for版本一致；不得在此法中使用i*i。
 - 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入 5 2 bytes · 最後有 LF 換行	STDOUT · 範例輸出 1 1 2 4 3 9 4 16 5 25 22 bytes · 包含程式實際輸出的換行與空白
---	--

跳至主要內容

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 從square=1開始。
- 2 每次先印目前平方。
- 3 把下一個奇數加進square。
- 4 奇數每次增加2。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

目前i	square	下一個差值
1	1	3
2	4	5
3	9	7
4	16	9
5	25	11

為什麼這個解法成立？

$(i+1)^2-i^2=2i+1$ ，奇數差每回合增加2，因此累加後仍保持square= i^2 。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

⌵

⌶

跳至主要內容

行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
```



```
int n, i = 1;
long long square = 1, odd = 3;
if (scanf("%d", &n) != 1) { puts("ERR_INPUT"); return 1; }
if (n < 1 || n > 20) { puts("ERR_RANGE"); return 1; }
for (; i <= n; odd += 2) {
    printf("%d %lld\n", i, square);
    i++;
    square += odd;
}
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s08_square_odd.c -o s08_square_odd
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S08-T01

1~5平方

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
5	1 1 2 4 3 9 4 16 5 25
2 bytes · 最後有 LF 換行	22 bytes · 包含程式實際輸出的換行與空白

—

S08-T02

單列

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
跳至主要內容 1	1 1

2 bytes · 最後有 LF 換行

4 bytes · 包含程式實際輸出的換行與空白

S08-T03 最大合法列數

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

20

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
10 100
11 121
12 144
13 169
14 196
15 225
16 256
17 289
18 324
19 361
20 400

119 bytes · 包含程式實際輸出的換行與空白

S08-T04 拒絕零列

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

跳至主要內容 : 敗

exit 1 ✓

L 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入
x
2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出
ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 本HTML以long long保存累加狀態，輸出格式沿用C06而非原教材固定欄寬。
- 原Ch6 pp.45–46提供square3.c；整合版C06易錯提醒提及此方法。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

while、do、for 與 continue 的執行次數

while

do

continue

對應整合版：第 29、44、55 頁 · 1 份程式 · 4 組測試

來源對照：整合版 p.29,44,55 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01 完整題目

先確認規格，再寫程式

輸入 n ，量測相同計數條件下 while、do 和 for 的本體次數，再用 for 跳過偶數，輸出 $0 \sim n-1$ 中的奇數。

輸入規格

一個整數 n ， $0 \sim 20$ 。

輸出規格

印 While、Do、For 的次數，接著印 Odd 序列。

規則與限制

- 三種迴圈各自從 $i=0$ 、 $count=0$ 開始。
- do 先執行 $i++$ 與 $count++$ ，再檢查 $i < n$ ，所以 $n=0$ 時仍做一次。
- continue 遇到偶數時直接到 for 更新式，不在本體額外增加 i 。
- While、Do、For 各使用「標籤: 次數」獨立一行；Odd: 後每個奇數前加一個空白，最後換行。三種計數互不共用累計結果。
- 共同錯誤處理：讀取數值失敗印 ERR_INPUT；數值可讀取但超出本題輸入範圍印 ERR_RANGE，皆以結束碼 1 停止。正常完成以 0 結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

0

2 bytes · 最後有 LF 換行

[跳至主要內容](#)

STDOUT · 範例輸出

While: 0
Do: 1
For: 0
Odd:

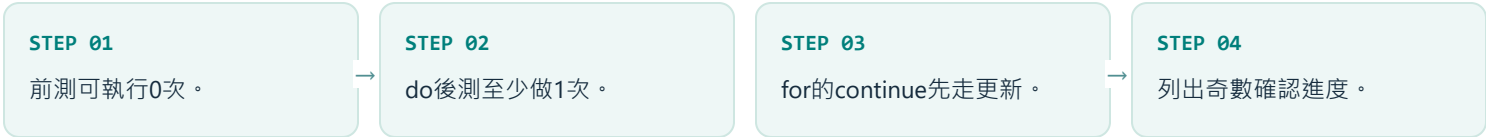
27 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 前測可執行0次。
- 2 do後測至少做1次。
- 3 for的continue先走更新。
- 4 列出奇數確認進度。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

i	continue?	輸出
0	是	無
1	否	1
2	是	無
3	否	3
4	是	無

為什麼這個解法成立？

三個次數由是否在第一輪之前檢查條件決定；for更新不因continue被跳過。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

跳至主要內容

行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
```

```
int n, i = 0, count = 0;
if (scanf("%d", &n) != 1) { puts("ERR_INPUT"); return 1; }
if (n < 0 || n > 20) { puts("ERR_RANGE"); return 1; }
while (i < n) { count++; i++; }
printf("While: %d\n", count);
i = 0; count = 0;
do { count++; i++; } while (i < n);
printf("Do: %d\n", count);
count = 0;
for (i = 0; i < n; i++) count++;
printf("For: %d\nOdd:", count);
for (i = 0; i < n; i++) {
    if (i % 2 == 0) continue;
    printf(" %d", i);
}
putchar('\n');
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s09_loop_counts.c -o s09_loop_counts
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

4 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

S09-T01 前測與後測的差異

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0	While: 0 Do: 1 For: 0 Odd:
2 bytes · 最後有 LF 換行	27 bytes · 包含程式實際輸出的換行與空白

跳至主要內容 ue跳過偶數

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

5

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

While: 5
Do: 5
For: 5
Odd: 1 3

31 bytes · 包含程式實際輸出的換行與空白

—

S09-T03 僅有i=0 · 奇數序列空

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

1

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

While: 1
Do: 1
For: 1
Odd:

27 bytes · 包含程式實際輸出的換行與空白

—

S09-T04 最大合法n

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

20

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

While: 20
Do: 20
For: 20
Odd: 1 3 5 7 9 11 13 15 17 19

55 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 跳至主要內容 while且把i++留在continue之後，可能不再前進；這裡不執行該錯誤版本。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

有意的空本體與讀到行尾

空敘述

EOF

getchar

對應整合版：第 54 頁 · 1 份程式 · 4 組測試

來源對照：整合版 p.54 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

先執行教材的有意空本體倒數，輸出最終i；接著略過輸入的第一行，再印下一個字元的數值，觀察空本體如何在條件內完成讀取。

輸入規格

任意ASCII文字；最多讀到第一個換行，接著再讀一個byte。

輸出規格

印AfterEmpty: 3；再印NextCode: 數值或Next: EOF。

- 規則與限制
- for(i=10;i>3;i--)本體使用空區塊。
 - 略過行尾必須同時檢查EOF，避免空檔無法終止。
 - 此題不執行while(i>0);而沒有更新的無限迴圈。
 - 若第一行以換行結束，該換行會被消耗，NextCode觀察的是其後第一個byte；若已無資料則印Next: EOF。串流讀取錯誤印ERR_INPUT，結束碼1；正常EOF結束碼0。

輸入 / 輸出範例

STDIN · 範例輸入

abc
Q

6 bytes · 最後有 LF 換行

STDOUT · 範例輸出

AfterEmpty: 3
NextCode: 81

27 bytes · 包含程式實際輸出的換行與空白

輸，跳至主要內容²。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 空for在更新式中前進。
- 2 getchar在while條件內讀取。
- 3 換行或EOF停止略過。
- 4 再讀下一個byte。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

輸入區段	處理
a b c	被條件中的getchar消耗
換行	終止略過
Q	下一次getchar取得81

為什麼這個解法成立？

空本體仍可正確工作，只要前進動作與停止條件放在條件 / 更新式內。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s10_empty_loop.c

補齊後的完整程式 · 16 行 · 長行可橫向捲動

#include <stdio.h>

跳至主要內容

for (i = 10, i > 3; i--) { }

printf("AfterEmpty: %d\n", i);

while ((ch = getchar()) != EOF && ch != '\n') { }

```
if (ferror(stdin)) { puts("ERR_INPUT"); return 1; }
ch = getchar();
if (ch == EOF) {
    if (ferror(stdin)) { puts("ERR_INPUT"); return 1; }
    puts("Next: EOF");
} else {
    printf("NextCode: %d\n", ch);
}
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s10_empty_loop.c -o s10_empty_loop
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

4 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S10-T01 略過第一行後讀到Q，ASCII=81

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
abc Q 6 bytes · 最後有 LF 換行	AfterEmpty: 3 NextCode: 81 27 bytes · 包含程式實際輸出的換行與空白

—

S10-T02 空檔

exit 0 ✓

本 HTML 新增測試 · 0 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
(空輸入檔 · 直接 EOF) 跳至主要內容 不是空白行	AfterEmpty: 3 Next: EOF 24 bytes · 包含程式實際輸出的換行與空白

—

S10-T03 未換行即EOF

exit 0 ✓

STDIN · 完整輸入	STDOUT · 完整預期輸出
abc	AfterEmpty: 3 Next: EOF
3 bytes · 最後沒有換行 · 檔案直接結束	24 bytes · 包含程式實際輸出的換行與空白

—

S10-T04 第一行為空行

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
Z	AfterEmpty: 3 NextCode: 90
2 bytes · 最後沒有換行 · 檔案直接結束	27 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 字元數值測試限定ASCII相容環境；若不是ASCII，數值觀察可能不同。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

整數常數進位與輸入輸出格式

八進位

十六進位

scanf

對應整合版：第 59、62 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.59,62 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

觀察原始碼中10、010、0x10的差異，再分別以十進位、八進位、十六進位格式讀取三個unsigned值並全部用十進位印出。

輸入規格

依序三個token：十進位、八進位、十六進位；每個解讀後的值限0 ~ 10000。

輸出規格

印Literals: 10 8 16，及Read: 三個十進位數值。

規則與限制

- 第二個token使用%o、第三個使用%x；輸入須符合各自的合法進位文字。
- 仍沿用教學scanf輸入契約，不是拒絕所有尾隨垃圾的完整剖析器。
- 第二行使用Read: d o h，其中三個值都是解讀後的十進位值，值之間以單一空白分隔；範例16 20 10會讀出16 16 16。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

10 10 10

9 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Literals: 10 8 16

Read: 10 8 16

32 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

跳至主要內容

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1
- 先分清原始碼常數前綴。
- 2
- scanf格式決定讀取進位。
- 3
- 最後統一用%u印出。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

文字10	規則	數值
原始碼10	十進位	10
原始碼010	八進位	8
原始碼0x10	十六進位	16

為什麼這個解法成立？

相同文字形狀不一定代表相同值；程式直接指定每個欄位的進位解讀。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s11_integer_io.c

補齊後的完整程式 · 13 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    unsigned int decimal, octal, hexadecimal;
    if (scanf("%u%o%u", &decimal, &octal, &hexadecimal) != 3) {
        跳至主要內容 ↩_INPUT"); return 1;
    }
    if (decimal > 10000U || octal > 10000U || hexadecimal > 10000U) {
        puts("ERR_RANGE"); return 1;
    }
}
```

```
printf("Literals: %d %d %d\n", 10, 010, 0x10);
printf("Read: %u %u %u\n", decimal, octal, hexadecimal);
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s11_integer_io.c -o s11_integer_io
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S11-T01

相同字面字串使用不同進位讀取

exit 0 ✓

本 HTML 新增測試 · 9 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
10 10 10	Literals: 10 8 16 Read: 10 8 16
9 bytes · 最後有 LF 換行	32 bytes · 包含程式實際輸出的換行與空白

—

S11-T02

相同數值使用三種進位

exit 0 ✓

本 HTML 新增測試 · 11 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
255 377 ff	Literals: 10 8 16 Read: 255 255 255
11 bytes · 最後有 LF 換行	36 bytes · 包含程式實際輸出的換行與空白

—

S11-T03

零

exit 0 ✓

L 新增測試 · 6 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入	STDOUT · 完整預期輸出

`0 0 0`

6 bytes · 最後有 LF 換行

`Literals: 10 8 16
Read: 0 0 0`

30 bytes · 包含程式實際輸出的換行與空白

S11-T04

十進位讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

`x`

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

`ERR_INPUT`

10 bytes · 包含程式實際輸出的換行與空白

S11-T05

超出題目數值上限

exit 1 ✓

本 HTML 新增測試 · 10 input bytes · 完整比對

STDIN · 完整輸入

`10001 0 0`

10 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

`ERR_RANGE`

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 原始碼的08不是合法八進位常數，沒有把它放進可編譯程式中。
- 這裡沒有使用%i自動判斷進位。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

unsigned 回繞、整數提升與混合比較

unsigned

整數提升

wrap

對應整合版：第 60、69 頁 · 1 份程式 · 1 組測試

來源對照：整合版 p.60,69 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

不讀取輸入，示範UINT_MAX加1U回繞、int的-1與1U比較時的轉換，並在8-bit unsigned char環境中觀察250+10先得到260、轉回byte後得到4。

輸入規格

不需輸入；CHAR_BIT等實作條件由編譯環境決定。

輸出規格

印Wrap、MixedLess；支援指定byte條件時印PromotedSum與Narrowed，否則印ByteDemo: SKIPPED。

規則與限制

- 不得實際執行INT_MAX+1的有號溢位。
- 混合比較明示轉換後的unsigned值，重現原式-1<1U的語意。
- 小型整數示例用編譯期條件限制為UCHAR_MAX=255且int可容納260。
- Wrap輸出UINT_MAX+1U的結果；MixedLess輸出(unsigned int)(-1)<1U的0/1結果。指定byte條件成立時，PromotedSum印提升後的250+10，Narrowed印再轉成unsigned char的結果。

輸入 / 輸出範例

以下輸出為本次 Linux / GCC 的**完整觀察值**，不是所有電腦都必須相同的固定答案。Wrap 與 MixedLess 必須為0；若編譯條件成立，應印260與4，否則允許 ByteDemo: SKIPPED。兩條輸出路徑都由程式的條件編譯明示。

STDIN · 範例輸入

跳至主要內容

OF)

STDOUT · 本次觀察

Wrap: 0

MixedLess: 0

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 在unsigned中相加。
- 2 按規則明示mixed比較轉換。
- 3 先加法再轉回unsigned char。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

階段	常見8-bit byte環境
UINT_MAX+1U	0
-1轉unsigned再比1U	假
a+b	int 260
轉成unsigned char	4

為什麼這個解法成立？

原投影片的各種超界現象由不同規則導致；本程式不執行沒有固定輸出的有號溢位。

本題資料規模已限制；重點為所示控制流程與型別行為。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

s12_unsigned_rules.c

補齊後的完整程式 · 19 行 · 長行可橫向捲動

```
#include <stdio.h>
#include <limits.h>
int main(void) {
    unsigned int u = UINT_MAX;
    u = u + 1U;
    int s = -1;
    unsigned int converted = (unsigned int)s;
    printf("Wrap: %u\n", u);
    printf("MixedLess: %d\n", converted < 1U);
    #if UCHAR_MAX == 255 && INT_MAX >= 260
        unsigned char a = 250, b = 10;
        int sum = a + b;
        unsigned char narrowed = (unsigned char)sum;
        printf("PromotedSum: %d\nNarrowed: %u\n", sum, (unsigned int)narrowed);
    #else
        puts("ByteDemo: SKIPPED");
    #endif
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s12_unsigned_rules.c -o s12_unsigned_rules
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

1 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

Wrap 與 MixedLess 必須為0；若編譯條件成立，應印260與4，否則允許 ByteDemo: SKIPPED。兩條輸出路徑都由程式的條件編譯明示。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— S12-T01 平台分支檢查：不將byte假設強加到其他實作

exit 0 ✓

L 新增測試 · 0 input bytes · 平台判準

— 跳至主要內容

STDIN · 完整輸入

STDOUT · 本次觀察 (非跨平台固定答案)

(空輸入檔，直接 EOF)	Wrap: 0
	MixedLess: 0
0 bytes · 0 bytes · 不是空白行	PromotedSum: 260
	Narrowed: 4
	50 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 本題不是把所有型別超界都統稱溢位；無號算術、提升與縮窄轉換分開。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

先轉型再除法：平均值與零筆資料

cast

整數除法

零分母

對應整合版：第 70、81、93 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.70,81,93 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01 完整題目

先確認規格，再寫程式

輸入sum、count，對比(double)(sum/count)與(double)sum/count；count=0時輸出NO_DATA，不做除法。此題補成第93頁平均值除錯題的完整程式。

輸入規格

sum為-10000~10000的整數；count為0~1000。

輸出規格

正常印AfterDivision與BeforeDivision (2位小數) ；
count=0印NO_DATA。

規則與限制

- 只比較轉型時機，不把sum重新由資料串流算出。
- 讀取或範圍錯誤印ERR_INPUT或ERR_RANGE，結束碼1。
- AfterDivision先做整數sum/count，再將結果轉成double；BeforeDivision在除法之前將sum轉成double。正常兩行皆顯示兩位小數；NO_DATA為正常的0結束碼。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

3 2

4 bytes · 最後有 LF 換行

STDOUT · 範例輸出

AfterDivision: 1.00

BeforeDivision: 1.50

41 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

輸, ²。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 先判定count是否為0。
- 2 先除法會先截斷小數。
- 3 先轉型才以浮點相除。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

運算	3/2中間值	最後結果
先整數除法	1	1.00
先轉double	3.0/2.0	1.50

為什麼這個解法成立？

浮點型別不能復原之前整數除法已捨棄的小數部分。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s13_cast_mean.c

補齊後的完整程式 · 13 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int sum, count;
    if (scanf("%d %d", &sum, &count) != 2) { puts("ERR_INPUT"); return 1; }
    if (sum < -10000 || sum > 10000 || count < 0 || count > 1000) {
        puts("ERR_RANGE"); return 1;
    }
    // 跳至主要內容
    if (count == 0) { puts("NO_DATA"); return 0; }
    double after = (double)(sum / count);
    double before = (double)sum / count;
    printf("AfterDivision: %.2f\nBeforeDivision: %.2f\n", after, before);
}
```

```
    return 0;
}
```

單題編譯（在包含本檔的資料夾執行）：

```
gcc -std=c11 -Wall -Wextra -Wpedantic s13_cast_mean.c -o s13_cast_mean
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

- S13-T01

第93頁的3/2

exit 0 ✓
- 本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
3 2	AfterDivision: 1.00 BeforeDivision: 1.50
4 bytes · 最後有 LF 換行	41 bytes · 包含程式實際輸出的換行與空白

- S13-T02

第70頁的7/2

exit 0 ✓
- 本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
7 2	AfterDivision: 3.00 BeforeDivision: 3.50
4 bytes · 最後有 LF 換行	41 bytes · 包含程式實際輸出的換行與空白

- S13-T03

負整數除法向零截斷

exit 0 ✓
- 本 HTML 新增測試 · 5 input bytes · 完整比對

跳至主要內容	STDOUT · 完整預期輸出
-7 2	

5 bytes · 最後有 LF 換行

AfterDivision: -3.00
BeforeDivision: -3.50

43 bytes · 包含程式實際輸出的換行與空白

S13-T04 零分母不運算

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

0 0

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

NO_DATA

8 bytes · 包含程式實際輸出的換行與空白

S13-T05 非法count

exit 1 ✓

本 HTML 新增測試 · 5 input bytes · 完整比對

STDIN · 完整輸入

1 -1

5 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 這是轉型順序問題，不是float或double精度不足。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

float、double、long double 的 I/O 與精度觀察

float

%lf

精度

對應整合版：第 66–68 頁 · 1 份程式 · 4 組測試

來源對照：整合版 p.66,67,68 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

分別讀入float、double與long double，使用正確格式輸出，再印出0.1+0.2的高精度觀察及只顯示兩位小數的結果。

輸入規格

三個有限實數f d ld，各在-10000 ~ 10000。

輸出規格

印Float、Double、LongDouble (6位小數)、Sum17 (17位有效數字) 與Sum2 (2位小數)。

規則與限制

- scanf使用%f、%lf、%Lf；printf的float/double用%f，long double用%Lf。
- Sum17是本環境觀察，不預設所有C實作都印出相同末位。
- 輸入錯誤或不在有限範圍內印ERR_INPUT或ERR_RANGE。
- 五個欄位各自一行，格式皆為「標籤: 值」。Sum17與Sum2都觀察double sum=0.1+0.2，不是輸入f、d、ld的加總。正常結束碼0，輸入或範圍錯誤結束碼1。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

以下輸出為本次 Linux / GCC 的完整觀察值，不是所有電腦都必須相同的固定答案。本組測試使用可精確表示的1.25及0，前三行對照六位小數；Sum2 必須為0.30。Sum17 須為有限數且與0.3的差小於10^-12，不將某一平台的最後一位寫成普遍答案。此誤差門檻是本測試所選，而非語言對所有浮點實作的保證。

跳至主要內容

1.25 1.25 1.25

STDOUT · 本次觀察

Float: 1.250000
Double: 1.250000

15 bytes · 最後有 LF 換行

LongDouble: 1.250000
Sum17: 0.30000000000000004
Sum2: 0.30

92 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 按照目標物件選讀取格式。
- 2 確認三個值都是合法範圍。
- 3 區分儲存值與輸出格式。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

輸出	常見binary64環境
Sum17	0.30000000000000004
Sum2	0.30
結論	格式化不改變內部儲存

為什麼這個解法成立？

讀取契約與輸出參數提升不同；以適合每個物件的格式才能可靠讀取。

本題資料規模已限制；重點為所示控制流程與型別行為。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

s14_floating_io.c

補齊後的完整程式 · 16 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    float f;
    double d;
    long double ld;
    if (scanf("%f %lf %Lf", &f, &d, &ld) != 3) { puts("ERR_INPUT"); return 1; }
    if (!(f >= -10000.0f && f <= 10000.0f) ||
        !(d >= -10000.0 && d <= 10000.0) ||
        !(ld >= -10000.0L && ld <= 10000.0L)) {
        puts("ERR_RANGE"); return 1;
    }
    double sum = 0.1 + 0.2;
    printf("Float: %.6f\nDouble: %.6f\nLongDouble: %.6Lf\n", f, d, ld);
    printf("Sum17: %.17g\nSum2: %.2f\n", sum, sum);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s14_floating_io.c -o s14_floating_io
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

4 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

本組測試使用可精確表示的1.25及0，前三行對照六位小數；Sum2 必須為0.30。Sum17 須為有限數且與0.3的差小於 10^{-12} ，不將某一平台的最後一位寫成普遍答案。此誤差門檻是本測試所選，而非語言對所有浮點實作的保證。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— S14-T01 精確二進位輸入與環境浮點觀察

exit 0 ✓

本 HTML 新增測試 · 15 input bytes · 平台判準

跳至主要內容

1.25 1.25 1.25

STDOUT · 本次觀察 (非跨平台固定答案)

Float: 1.250000

Double: 1.250000

15 bytes · 最後有 LF 換行

LongDouble: 1.250000
Sum17: 0.30000000000000004
Sum2: 0.30

92 bytes · 包含程式實際輸出的換行與空白

S14-T02 零輸入與固定觀察

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 平台判準

STDIN · 完整輸入

0 0 0

6 bytes · 最後有 LF 換行

STDOUT · 本次觀察 (非跨平台固定答案)

Float: 0.000000
Double: 0.000000
LongDouble: 0.000000
Sum17: 0.30000000000000004
Sum2: 0.30

92 bytes · 包含程式實際輸出的換行與空白

S14-T03 拒絕NaN

exit 1 ✓

本 HTML 新增測試 · 8 input bytes · 完整比對

STDIN · 完整輸入

nan 1 1

8 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

S14-T04 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

跳至主要內容

錯也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 以%.2f印成0.30不代表內部精確儲存十進位0.3。
- 本示例不執行scanf格式不匹配的錯誤版本。

教學輸入契約：數值 `token` 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

字元、跳脫字元與 toupper

char

ASCII

toupper

對應整合版：第 74、76、77 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.74,76,77 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

讀入一個ASCII字元，印出其數值、大寫版本與數字值；最後示範八進位跳脫、分開十六進位跳脫與tab輸出。

輸入規格

一個ASCII字元 (空白也可) ；立即EOF視為ERR_INPUT。

輸出規格

印Code、Upper、Digit；以及HexJoin: AB、Octal: S、Tab: A與B中間一個tab。

規則與限制

- getchar用int接收；先排除EOF再交給toupper或isdigit。
- 不是數字時Digit印-1；數字用ch-'0'轉數值。
- 字元數值與跳脫觀察限定ASCII相容環境。
- Upper使用toupper的結果；Code為原輸入byte的十進位值。只讀第一個byte，不消耗或解讀整行。正常結束碼0；輸入EOF或byte值大於127時分別印ERR_INPUT、ERR_RANGE並以1結束。
- HexJoin與Octal為固定示範文字；Tab一行的實際輸出為Tab: A\tB\n，其中\t和\n分別代表一個tab與換行，不是反斜線字母。

輸入 / 輸出範例

STDIN · 範例輸入

a

2 bytes · 最後有 LF 換行

跳至主要內容

STDOUT · 範例輸出

Code: 97
Upper: A
Digit: -1
HexJoin: AB
Octal: S
Tab: A B

58 bytes · 包含程式實際輸出的換行與空白

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 以int保存讀取結果。
- 2 使用ctype分類與轉大寫。
- 3 分開字串常值終止hex跳脫。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

項目	輸入a
getchar	97
toupper	A
Digit	-1
"\x41" "B"	AB

為什麼這個解法成立？

字元編碼、字串拼接與ctype使用契約各自明確，不依不安全的char型別保存EOF。

本題資料規模已限制；重點為所示控制流程與型別行為。

03 完整程式碼

每一份都可獨立編譯

s15_char_escape.c

補齊後的完整程式 · 13 行 · 長行可橫向捲動

跳至主要內容

```
:  
#include <ctype.h>  
int main(void) {  
    int ch = getchar();
```

```
if (ch == EOF) { puts("ERR_INPUT"); return 1; }
if (ch > 127) { puts("ERR_RANGE"); return 1; }
printf("Code: %d\nUpper: %c\n", ch, toupper(ch));
printf("Digit: %d\n", isdigit(ch) ? ch - '0' : -1);
puts("HexJoin: " "\x41" "B");
printf("Octal: %c\n", '\123');
printf("Tab: A\tB\n");
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s15_char_escape.c -o s15_char_escape
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S15-T01 小寫轉大寫

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
a 2 bytes · 最後有 LF 換行	Code: 97 Upper: A Digit: -1 HexJoin: AB Octal: S Tab: A B 58 bytes · 包含程式實際輸出的換行與空白

—

S15-T02 數字轉數值

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
跳至主要內容 2 bytes · 最後有 LF 換行	Code: 55 Upper: 7 Digit: 7 HexJoin: AB

STDIN · 完整輸入

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Octal: S
Tab: A B

57 bytes · 包含程式實際輸出的換行與空白

–

S15-T03 空白不被略過

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Code: 32
Upper:
Digit: -1
HexJoin: AB
Octal: S
Tab: A B

58 bytes · 包含程式實際輸出的換行與空白

–

S15-T04 大寫保持不變

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

A

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Code: 65
Upper: A
Digit: -1
HexJoin: AB
Octal: S
Tab: A B

58 bytes · 包含程式實際輸出的換行與空白

–

S15-T05 立即EOF

exit 1 ✓

本 HTML 新增測試 · 0 input bytes · 完整比對

STDIN · 完整輸入

(勿輸入檔，直接 EOF)

跳至主要內容

0 bytes · 0 bytes · 不是空白行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- "\x41B"不是字母A接B，因為B也屬十六進位數字；本程式使用相鄰字串拼接。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

scanf 後殘留的換行與下一個命令

scanf

getchar

輸入緩衝

對應整合版：第 75 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.75 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01 完整題目

先確認規格，再寫程式

先用scanf讀整數，再用getchar觀察下一個未消耗字元，最後用scanf(" %c")讀下一個非空白命令。

輸入規格

一個-10000 ~ 10000的整數、至少一個後續ASCII字元，以及一個仍可供讀取的非空白ASCII命令。

輸出規格

印Number、Next (LF/SPACE/TAB/EOF或Code)、Command。

規則與限制

- getchar不略過空白，讀到的字元會被消耗。
- " %c"前的空白會略過連續空白，可能跨行。
- 缺少命令時在已印出的觀察之後印ERR_INPUT並結束。
- Next觀察到換行、空白、tab或EOF時分別印Next: LF、Next: SPACE、Next: TAB、Next: EOF；其他值印Next: Code N，N為byte十進位值。最後成功讀到命令才印Command: X。
- 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

123
Q

6 bytes · 最後有 1 LF 換行

跳至主要內容

STDOUT · 範例輸出

Number: 123
Next: LF
Command: Q

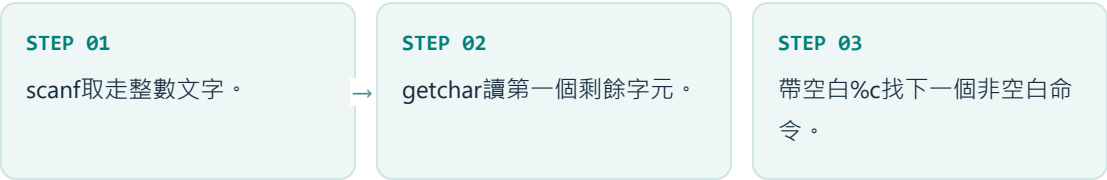
32 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1
- scanf取走整數文字。
- 2
- getchar讀第一個剩餘字元。
- 3
- 帶空白%c找下一個非空白命令。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

串流	動作
123\nQ\n	scanf讀走123
\nQ\n	getchar得到換行
Q\n	scanf(" %c")得到Q

為什麼這個解法成立？

分別觀察每次呼叫前的剩餘串流，可以預測下一次讀取會拿到什麼。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s16_input_buffer.c

補齊後的完整程式 · 17 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int number;
    跳至主要內容 ;
    if (scanf("%d", &number) != 1) { puts("ERR_INPUT"); return 1; }
    if (number < -10000 || number > 10000) { puts("ERR_RANGE"); return 1; }
    printf("Number: %d\n", number);
}
```

```
ch = getchar();
if (ch == '\n') puts("Next: LF");
else if (ch == ' ') puts("Next: SPACE");
else if (ch == '\t') puts("Next: TAB");
else if (ch == EOF) puts("Next: EOF");
else printf("Next: Code %d\n", ch);
if (scanf(" %c", &command) != 1) { puts("ERR_INPUT"); return 1; }
printf("Command: %c\n", command);
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s16_input_buffer.c -o s16_input_buffer
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

— S16-T01 教材殘留換行

exit 0 ✓

本 HTML 新增測試 · 6 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
123 Q 6 bytes · 最後有 LF 換行	Number: 123 Next: LF Command: Q 32 bytes · 包含程式實際輸出的換行與空白

— S16-T02 多個空白

exit 0 ✓

本 HTML 新增測試 · 8 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入

123 Q

8 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Number: 123
Next: SPACE
Command: Q

35 bytes · 包含程式實際輸出的換行與空白

S16-T03 **getchar先消耗Q** exit 0 ✓

本 HTML 新增測試 · 7 input bytes · 完整比對

STDIN · 完整輸入

123Q R

7 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Number: 123
Next: Code 81
Command: R

37 bytes · 包含程式實際輸出的換行與空白

S16-T04 **缺少後續命令** exit 1 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

123

3 bytes · 最後沒有換行 · 檔案直接結束

STDOUT · 完整預期輸出

Number: 123
Next: EOF
ERR_INPUT

32 bytes · 包含程式實際輸出的換行與空白

S16-T05 **首筆讀取失敗** exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入

x

跳至主要內容
: 換行

STDOUT · 完整預期輸出

ERR_INPUT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05 易錯提醒與檢查重點

- 本題是刻意觀察「先消耗一個字元」；一般讀命令不一定需要中間那次`getchar`。
- 不使用`fflush(stdin)`處理殘留輸入。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

[跳至主要內容](#)

typedef、sizeof 與精確寬度型別

typedef

sizeof

stdint

對應整合版：第 80 頁 · 1 份程式 · 1 組測試

來源對照：整合版 p.80 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

不讀取輸入，驗證Cents只是long long的別名、sizeof(char)=1，且固定型別的sizeof(i++)不會自增i；若環境提供uint32_t，使用PRIu32印出42。

輸入規格

不需輸入。

輸出規格

印AliasSameSize、NegativeAllowed、CharSize、AfterSizeof與U32。

規則與限制

- Cents先存-1，說明typedef沒有自動加入非負限制。
- sizeof的值先存成size_t但不把平台int大小當成固定答案。
- uint32_t以UINT32_MAX條件檢查是否存在，不假定所有實作都有。
- AliasSameSize為sizeof(Cents)==sizeof(long long)的0/1值；NegativeAllowed為balance=-1之後balance<0的0/1值；CharSize印sizeof(char)。
- i初值為7；先取得sizeof(i++)但不使用其數值，再印AfterSizeof: i。若UINT32_MAX有定義則印U32: 42，否則印U32: UNAVAILABLE；正常結束碼0。

輸入 / 輸出範例

以下輸出為本次 Linux / GCC 的**完整觀察值**，不是所有電腦都必須相同的固定答案。別名大小相同、仍可為負、sizeof(char)=1及未執行i++是固定判準；精確32位元型別存在時印42，否則允許 UNAVAILABLE。

跳至主要內容

(空輸入檔，直接 EOF)

STDOUT · 本次觀察

AliasSameSize: 1
NegativeAllowed: 1


```
0 bytes · 0 bytes · 不是空白行
```

```
CharSize: 1
AfterSizeof: 7
U32: 42

71 bytes · 包含程式實際輸出的換行與空白
```

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 typedef建立型別別名。
- 2 sizeof固定型別不求值。
- 3 平台支援才使用uint32_t。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

表達式	結果
sizeof(Cents)==sizeof(long long)	1
balance=-1	允許
sizeof(i++)後	i仍為7
uint32_t	視環境提供情形

為什麼這個解法成立？

型別別名不改變底層值域；sizeof的固定型別操作不執行i++。

本題資料規模已限制；重點為所示控制流程與型別行為。

跳至主要內容

03 完整程式碼

每一份都可獨立編譯

s17_typedef_sizeof.c

補齊後的完整程式 · 21 行 · 長行可橫向捲動

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
typedef long long Cents;
int main(void) {
    Cents balance = -1;
    int i = 7;
    size_t size = sizeof(i++);
    (void)size;
    printf("AliasSameSize: %d\n", sizeof(Cents) == sizeof(long long));
    printf("NegativeAllowed: %d\n", balance < 0);
    printf("CharSize: %zu\n", sizeof(char));
    printf("AfterSizeof: %d\n", i);
#ifdef UINT32_MAX
    uint32_t value = 42;
    printf("U32: %" PRIu32 "\n", value);
#else
    puts("U32: UNAVAILABLE");
#endif
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s17_typedef_sizeof.c -o s17_typedef_sizeof
```

刻意保留的教學警告：Clang 會對 `sizeof(i++)` 發出未求值運算式警告。自動測試只對本題使用 `-Wno-error=unevaluated-expression`，保留警告而不將其升級成錯誤；沒有關閉其他警告。一般程式不應在 `sizeof` 中放無作用的自增。

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04 測試資料

1 組完整資料

核對輸出、結束碼與 `stderr`。全部輸入 / 輸出已內嵌，可個別下載。

別名大小相同、仍可為負、`sizeof(char)=1`及未執行`i++`是固定判準；精確32位元型別存在時印42，否則允許

跳至主要內容

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 0 bytes · 不是空白行

STDOUT · 本次觀察 (非跨平台固定答案)

AliasSameSize: 1
NegativeAllowed: 1
CharSize: 1
AfterSizeof: 7
U32: 42

71 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 本題沒有VLA；不能將sizeof(i++)不自增的結論無條件推廣至所有變動長度陣列運算。
- 負數是否允許需要額外驗證，不是typedef會替你處理。
- 本題的 sizeof(i++) 刻意展示未求值運算式。Clang 會發出 unevaluated-expression 警告；自動測試只對本題把這個警告從錯誤降級，仍保留警告紀錄。一般程式請避免在 sizeof 中寫無作用的自增。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

安全的 unsigned 倒數

unsigned

倒數

除錯

對應整合版：第 93 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.93 (原題 / 語意依投影片及備忘錄)

原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

輸入 n ，依序輸出 $n-1$ 到 0 ； $n=0$ 時不處理任何項目。修正原題 unsigned $i \geq 0$ 永遠為真的問題。

輸入規格

一個整數 n ， $0 \sim 20$ 。

輸出規格

輸出 Reverse: 後接倒數項目，各相隔一個空白； $n=0$ 只印 Reverse: 及換行。

規則與限制

- 先讀成 int 以驗證非負，再轉 unsigned。
- 迴圈狀態從 n 開始，只在 $i > 0$ 時印 $i-1$ ，因此不會先算 $n-1$ 造成 $n=0$ 下溢。
- 每個倒數項目前都有一個空白，不在行尾另加空白；最後一定換行。
- 共同錯誤處理：讀取數值失敗印 ERR_INPUT；數值可讀取但超出本題輸入範圍印 ERR_RANGE，皆以結束碼 1 停止。正常完成以 0 結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

3

2 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Reverse: 2 1 0

15 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

跳至主要內容

02 解題說明

理解狀態如何改變



核心思考與步驟

- 1 由n而非n-1初始化。
- 2 只在i>0時進本體。
- 3 處理i-1後再遞減。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

i	輸出	下一個i
3	2	2
2	1	1
1	0	0
0	不進本體	停止

為什麼這個解法成立？

本體只在i為正時運作，i-1合法；遞減到0後條件失敗。

本題資料規模已限制；重點為所示控制流程與型別行為。

03 完整程式碼

每一份都可獨立編譯

s18_reverse_unsigned.c

補齊後的完整程式 · 13 行 · 長行可橫向捲動

#include <stdio.h>

跳至主要內容

1 if (scanf("%d", &input) != 1) { puts("ERR_INPUT"); return 1; }
2 if (input < 0 || input > 20) { puts("ERR_RANGE"); return 1; }
3 unsigned int n = (unsigned int)input;
4 printf("Reverse:");

```
for (unsigned int i = n; i > 0; --i) {
    printf(" %u", i - 1U);
}
putchar('\n');
return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s18_reverse_unsigned.c -o s18_reverse_unsigned
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S18-T01

3個項目

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
3	Reverse: 2 1 0
2 bytes · 最後有 LF 換行	15 bytes · 包含程式實際輸出的換行與空白

—

S18-T02

空序列不得下溢

exit 0 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
0	Reverse:
2 bytes · 最後有 LF 換行	9 bytes · 包含程式實際輸出的換行與空白

—

跳至主要內容

L 新增測試 · 2 input bytes · 完整比對

exit 0 ✓

STDIN · 完整輸入

1

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Reverse: 0

11 bytes · 包含程式實際輸出的換行與空白

–

S18-T04

上界

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

20

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

Reverse: 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3

59 bytes · 包含程式實際輸出的換行與空白

–

S18-T05

拒絕負n

exit 1 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_RANGE

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 沒有執行原本可能不停止的unsigned i >= 0迴圈。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

修正空本體與閉區間接受條件

除錯 空本體 閉區間

對應整合版：第 93 頁 · 1 份程式 · 5 組測試
來源對照：整合版 p.93 (原題 / 語意依投影片及備忘錄)
原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01 完整題目

先確認規格，再寫程式

將第93頁兩個除錯片段補成完整程式：先判斷score是否落在0 ~ 100 (含端點)，再由i=0遞增到3並逐次印出。

輸入規格

一個整數score，-1000 ~ 1000；其中只有0 ~ 100屬可接受成績。

輸出規格

印ACCEPT或REJECT；再印Step: 1、Step: 2、Step: 3與Final: 3。

- 規則與限制
- 接受判斷使用score >= 0 && score <= 100。
 - 移除while後多餘分號，i++確實位於迴圈本體。
 - 拒絕成績REJECT為正常分類，不是程式錯誤，仍執行迴圈。
 - 第一行分類與其後四行迴圈輸出分開；score在允許輸入範圍內但不屬0 ~ 100時，輸出REJECT仍正常結束。
 - 共同錯誤處理：讀取數值失敗印ERR_INPUT；數值可讀取但超出本題輸入範圍印ERR_RANGE，皆以結束碼1停止。正常完成以0結束；每一行輸出均以換行結尾。

輸入 / 輸出範例

STDIN · 範例輸入

150

4 bytes · 最後有 LF 換行

跳至主要內容

STDOUT · 範例輸出

REJECT
Step: 1
Step: 2
Step: 3
Final: 3

40 bytes · 包含程式實際輸出的換行與空白

輸入區不包含提示文字。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 用兩個比較合併閉區間。
- 2 刪除while後的空敘述。
- 3 本體每次確實增加i。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

i初值	本體後i	是否續做
0	1	是
1	2	是
2	3	否

為什麼這個解法成立？

迴圈每次減少距離終止條件的差距1；三次後i=3，條件為假。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s19_repair_basics.c

補齊後的完整程式 · 14 行 · 長行可橫向捲動

```
#include <stdio.h>
int main(void) {
    int score, i = 0;
    while (scanf("%d", &score) != 1) { puts("ERR_INPUT"); return 1; }
    跳至主要內容 while (score < -1000 || score > 1000) { puts("ERR_RANGE"); return 1; }
    while (i < 3) {
        if (score < 0 && score <= 100) puts("ACCEPT");
        else puts("REJECT");
        i++;
    }
}
```

```
        printf("Step: %d\n", i);
    }
    printf("Final: %d\n", i);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s19_repair_basics.c -o s19_repair_basics
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

- S19-T01

原題要求的score=150

exit 0 ✓
- 本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
150	REJECT
	Step: 1
	Step: 2
	Step: 3
	Final: 3
4 bytes · 最後有 LF 換行	40 bytes · 包含程式實際輸出的換行與空白

- S19-T02

含下界

exit 0 ✓
- 本 HTML 新增測試 · 2 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ACCEPT
Step: 1
Step: 2
Step: 3
Final: 3

40 bytes · 包含程式實際輸出的換行與空白

S19-T03 含上界

exit 0 ✓

本 HTML 新增測試 · 4 input bytes · 完整比對

STDIN · 完整輸入

100

4 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ACCEPT
Step: 1
Step: 2
Step: 3
Final: 3

40 bytes · 包含程式實際輸出的換行與空白

S19-T04 界外

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

REJECT
Step: 1
Step: 2
Step: 3
Final: 3

40 bytes · 包含程式實際輸出的換行與空白

S19-T05 讀取失敗

exit 1 ✓

本 HTML 新增測試 · 2 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入

STDOUT · 完整預期輸出

x
2 bytes · 最後有 LF 換行

ERR_INPUT
10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 原始錯誤片段保留在解題描述中，不放進可執行路徑。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

跳至主要內容

單行訊息長度：原 length.c 的完整化

getchar

長度

EOF

對應整合版：第 74、76、77 頁 · 1 份程式 · 5 組測試

來源對照：整合版 p.74,76,77 (原題 / 語意依投影片及備忘錄) ；另參照原 Chapter 7 Basic Types.pptx 第71–73頁的 length.c 。原投影片主要是語法片段 / 觀念示例；本 HTML 新增明確輸入輸出、main、必要標頭與測試。刻意保留錯誤行為的地方均另行說明，不把可能無限迴圈或未定義行為當成固定輸出。

01

完整題目

先確認規格，再寫程式

只計算一行訊息長度，包含空白與標點，不包含結尾換行。此題將原Ch7 length.c / length2.c的核心功能獨立保留，與C14的分類統計區分。

輸入規格

一行ASCII文字，最多4096 bytes；換行或EOF結束。

輸出規格

輸出Your message was N character(s) long.；超長印ERR_LIMIT。

- 規則與限制
- 使用int接getchar，加入原片段未完整處理的EOF與串流錯誤分支。
 - 不加入互動提示，讓標準輸入輸出可用檔案直接測試。
 - 不使用字串陣列，且本題不是UTF-8字數統計。
 - 長度0的空行或空檔都正常輸出0並以0結束；一旦讀到第4097個非換行byte就只印ERR_LIMIT並以1結束。串流讀取失敗則印ERR_INPUT並以1結束。

輸入 / 輸出範例

STDIN · 範例輸入

Brevity is the soul of wit.

28 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Your message was 27 character(s) long.

39 bytes · 包含程式實際輸出的換行與空白

輸，²。下載後可直接作為標準輸入檔；檔案結束代表 EOF，不需輸入「EOF」三個字母。

跳至主要內容

02

解題說明

理解狀態如何改變



核心思考與步驟

- 1 逐byte讀入。
- 2 在換行或EOF時停止。
- 3 成功讀入非換行值才累加。

固定範例追蹤

這是依指定範例整理的狀態表；切換步驟只改變標記，不是在瀏覽器中編譯或執行 C。

輸入	累計長度
H	1
i	2
!	3
EOF	不再增加

為什麼這個解法成立？

讀取一次只增加一次計數，且停止記號不計入長度。

本題資料規模已限制；重點為所示控制流程與型別行為。

03

完整程式碼

每一份都可獨立編譯

s20_message_length.c

補齊後的完整程式 · 12 行 · 長行可橫向捲動

#include <stdio.h>

跳至主要內容

size_t len = 0;

while ((ch = getchar()) != EOF && ch != '\n') {

if (len == 4096) { puts("ERR_LIMIT"); return 1; }

len++;

```
    }
    if (ferror(stdin)) { puts("ERR_INPUT"); return 1; }
    printf("Your message was %zu character(s) long.\n", len);
    return 0;
}
```

單題編譯 (在包含本檔的資料夾執行) :

```
gcc -std=c11 -Wall -Wextra -Wpedantic s20_message_length.c -o s20_message_length
```

完整測試方式與 Windows cmd.exe 操作範例請見使用與驗證說明。此頁為離線教材，不提供任意 C 程式的網頁執行環境。

04

測試資料

5 組完整資料

核對輸出、結束碼與 stderr。全部輸入 / 輸出已內嵌，可個別下載。

列印版僅列測試目的與結束碼；完整輸入與輸出請使用 HTML 展開或下載檔案，避免上限測試產生數千行列印內容。

—

S20-T01

原教材示範句

exit 0 ✓

本 HTML 新增測試 · 28 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
Brevity is the soul of wit.	Your message was 27 character(s) long.
28 bytes · 最後有 LF 換行	39 bytes · 包含程式實際輸出的換行與空白

—

S20-T02

空行

exit 0 ✓

本 HTML 新增測試 · 1 input bytes · 完整比對

STDIN · 完整輸入	STDOUT · 完整預期輸出
	Your message was 0 character(s) long.
1 bytes · 最後有 LF 換行	38 bytes · 包含程式實際輸出的換行與空白

—

S20-T03

空檔EOF

exit 0 ✓

L 新增測試 · 0 input bytes · 完整比對

跳至主要內容

STDIN · 完整輸入	STDOUT · 完整預期輸出

(空輸入檔 · 直接 EOF)

0 bytes · 0 bytes · 不是空白行

Your message was 0 character(s) long.

38 bytes · 包含程式實際輸出的換行與空白

S20-T04 沒有結尾換行

exit 0 ✓

本 HTML 新增測試 · 3 input bytes · 完整比對

STDIN · 完整輸入

Hi!

3 bytes · 最後沒有換行 · 檔案直接結束

STDOUT · 完整預期輸出

Your message was 3 character(s) long.

38 bytes · 包含程式實際輸出的換行與空白

S20-T05 超過長度上限

exit 1 ✓

本 HTML 新增測試 · 4,098 input bytes · 完整比對

STDIN · 完整輸入

aa

4,098 bytes · 最後有 LF 換行

STDOUT · 完整預期輸出

ERR_LIMIT

10 bytes · 包含程式實際輸出的換行與空白

錯誤分支的非零結束碼也屬預期結果，不代表測試失敗。測試器僅正規化 CRLF 為 LF，其餘空白需保持一致。

05

易錯提醒與檢查重點

- 本HTML將原題的char接getchar改成int並補EOF防護，屬明示補強；不是逐字照抄原碼。

教學輸入契約：數值 token 須可由程式的讀取型別表示；這些範例不是用來解析任意長數字或所有惡意輸入。詳細限制見共通輸入與測試規則。

HOW TO USE / VALIDATION

跳至主要內容

使用與驗證說明

先閱讀題目，再編譯單一程式，最後以同一份測試輸入核對結果。

這份 HTML 包含什麼？

共有 **16 個正式案例 + 20 個語法補充**，共 36 個主題、37 份可獨立執行的 C 程式，以及 262 組具體測試資料。C06 的 7 組資料分別測試 while 與 for，因此每輪執行 269 項「程式 × 資料」測試。

左側目錄與關鍵字搜尋可直接查找題目、語法、變數或程式片段。「隱藏解答」只提供自學遮蔽；完整解答仍在 HTML 中，不是保密考試頁面。即使完全離線，程式、輸入檔、預期輸出與整包 ZIP 均可下載。

頁面的「下一步」是固定範例追蹤表的逐列標記，不會執行 C 程式。真正的編譯與執行須在有 GCC / Clang 的電腦完成。

資料來源與新增範圍

C01–C16 沿用原教學資源包的完整題目與 17 份程式，已逐位元組比對相同；另外補充變數表、逐步追蹤、解題理由與測試。S01–S20 將投影片的語法片段 / 除錯練習補成明確題目及完整 main，均標為「語法補充」，不冒充原投影片本來就有的完整案例。

對應來源為 `C_Programming_Ch5-7_M365_Compatible.pptx`（101頁）及原資源包。S08 另參照原 Chapter 6 的 `square3.c`，S20 另參照原 Chapter 7 的 `length.c`。完整對照見來源索引。

單題編譯與執行

下載整包並解壓縮。每個 `.c` 都含自己的 main；不可將全部程式一起連結成一個可執行檔。以下假設目前位置為解壓縮後的資料夾。

Linux / macOS shell

```
gcc -std=c11 -Wall -Wextra -Wpedantic examples/c01_grade.c -o grade
./grade < tests/data/C01/C01-T01.in.txt
echo $?
```

Windows 命令提示字元 (cmd.exe)

跳至主要內容

```
gcc -std=c11 -Wall -Wextra -Wpedantic examples/c01_grade.c -o grade.exe
grade.exe < tests/data/C01/C01-T01.in.txt
```

```
echo %ERRORLEVEL%
```

Windows 命令供使用參考，本次未在 Windows 實測；上列 < 重導向寫法是 cmd.exe，不是 PowerShell 命令。需先具備編譯器；HTML 本身不會安裝工具。

使用自動測試器

```
python tests/run_tests.py --compiler gcc --report tests/local_gcc_report.json
python tests/run_tests.py --compiler clang --report tests/local_clang_report.json
python tests/run_tests.py --compiler gcc --sanitize --report tests/local_ubsan_report.json
```

測試器使用 Python 3.9 以上標準函式庫，不需額外 Python 套件。以暫存資料夾放編譯結果；不覆蓋教材程式或既有報告。`--sanitize` 需要編譯器提供對應支援。

S17 刻意使用 `sizeof(i++)` 展示未求值運算式，Clang 會發出教學上預期的警告。僅在該題加上 `-Wno-error=unevaluated-expression`，其餘警告仍依 `-Werror` 視為錯誤；完整警告保留於報告。

完整測試結果與判準

實測工具	程式編譯	程式×資料測試	說明
gcc	37 / 37	269 / 269	本輪未發現測試失敗
Clang 17.0.0	37 / 37	269 / 269	S17保留1項教學警告
GCC 14.2 + UBSan	37 / 37	269 / 269	本輪未發現測試失敗

三輪合計 807 次成功執行。以上是在本次 Linux x86-64 環境實際執行的結果；不是對所有輸入的形式證明，也不是 Windows、MSVC 或 Microsoft Office 相容性實測。

一般題目：精確比對

核對 **stdout** 完整內容、結束碼及空 **stderr**。只將 CRLF 正規化成 LF，其餘空白、換行與標點都保留。遇到 `ERR_*` 的題目，只要輸出和指定的結束碼相符，仍是成功通過錯誤分支測試。

平台判準：某台電腦的數字當作唯一答案
跳至主要內容

C12、S12、S14、S17 的部分測試採明確的欄位 / 關係判準。HTML 仍提供本次完整觀察輸出，資料檔名使用 `.observed-linux-gcc.out.txt`，不冒充跨平台固定預期輸出。S14 的浮點容許差是本測試選擇的門檻，並非語言對所有浮點實作的普遍保證。

空檔、空行、EOF 不一樣

0 bytes 的輸入檔會直接遇到 EOF。只含 `\n` 的檔案是一個空白行。EOF 是串流結束狀態，不是輸入字串「EOF」。「顯示跳脫字元」可辨認空白、tab 與最後的換行；下載保留的是原始資料，不是畫面上顯示的 JSON 跳脫寫法。

大型上限資料

1000筆、10000次等上限測試均提供**完整檔案內容**，沒有以省略符取代資料。HTML 中可在資料框內捲動或直接下載。列印時僅列出各測試目的與結束碼，完整輸入輸出以 HTML / 下載檔案為準。

共通輸入契約與限制

以 C11 模式編譯，教學主題主要對應第5–7章的 C99 內容。數值 token 假定可以由 scanf 對應的型別表示；範圍檢查不等於任意長數字的安全解析。程式也未一律拒絕已讀取數值後的所有額外字元；各題的停止條件與輸出規則以題目為準。

文字題以 **ASCII byte** 為規格，不是 UTF-8 中文字數計算。測試採預設 C locale，平台型別大小與浮點末位可能不同。手續費為教材假設，整數分帳本為教學實作，不是實際金融服務。

錯誤片段可能造成無限迴圈或未定義行為時，不直接把它當成可重現固定輸出的程式來執行；補充題會明示改寫方式，保留需要理解的語意差異。

SOURCE MAP / CONTENT INTEGRITY

來源與案例索引

頁碼指最新可編輯整合版，不是原始三份 PPTX 的頁碼。

主來源：`C_Programming_Ch5-7_M365_Compatible.pptx`；案例題目與17份原程式來自 `C_Programming_Ch5-7_Resources.zip`。本 HTML 以這些教材為依據整理，沒有將外部網路查詢混入來源內容。

保留與補充分開呈現。 C01–C16 的原程式逐位元組相同。新增的20題、解題追蹤及測試均清楚標示。平台量測題補上本次完整觀察值，而不是將省略號當成可執行答案。

[跳至主要內容](#)

編號	題目	整合版頁次	內容定位	程式檔名
C01	成績等第判定	10–12	正式案例 / 原程式保留	c01_grade.c
C02	分段手續費計算	13–16	正式案例 / 原程式保留	c02_commission.c
C03	四則計算器	20–22	正式案例 / 原程式保留	c03_calculator.c
C04	英文日期格式化	23–26	正式案例 / 原程式保留	c04_date.c
C05	找出足夠大的 2 次方容量	30–32	正式案例 / 原程式保留	c05_power2.c
C06	平方表：while 與 for 雙版本	33–36	正式案例 / 原程式保留	c06_squares_while.c, c06_squares_for.c
C07	以 0 結束的數列加總	38–40	正式案例 / 原程式保留	c07_sum.c
C08	計算十進位位數	41–43	正式案例 / 原程式保留	c08_digits.c
C09	質數判定與提早停止	45–47	正式案例 / 原程式保留	c09_prime.c
C10	座位編號產生器	48–50	正式案例 / 原程式保留	c10_seats.c
C11	距離感測資料清理	51–53	正式案例 / 原程式保留	c11_sensor.c
C12	量測目前環境的型別	63–65	正式案例 / 原程式保留	c12_type_lab.c
C13	RGB 影片的記憶體估算	71–73	正式案例 / 原程式保留	c13_rgb_memory.c
C14	一行文字的分類統計	76–79	正式案例 / 原程式保留	c14_text_stats.c
C15	互動式零用金帳本	83–87	正式案例 / 原程式保留	c15_ledger.c
C16	班級成績統計器	88–92	正式案例 / 原程式保留	c16_score_stats.c
S01	比較、連鎖不等式與否定的解析	6、9、28、93	語法片段 / HTML補齊	s01_comparison.c
S02	短路求值與被略過的自增	7、28	語法片段 / HTML補齊	s02_short_circuit.c
S03	大括號與 else 配對	8–9	語法片段 / HTML補齊	s03_dangling_else.c
S04	bool、條件運算子與獨立 if	9、17、28	語法片段 / HTML補齊	s04_bool_ternary.c
S05	switch 分派、共用標籤與 fall-through	18–19	語法片段 / HTML補齊	s05_switch_paths.c
S06	邏輯 AND 與低8位元運算	27	語法片段 / HTML補齊	s06_bitwise.c
S07	計數端點、逗號運算子與多變數更新	37	語法片段 / HTML補齊	s07_count_comma.c
S08	跳至主要內容 乘法的奇數累加平方表	33、34、37	語法片段 / HTML補齊	s08_square_odd.c
S09	while、do、for 與 continue 的執行次數	29、44、55	語法片段 / HTML補齊	s09_loop_counts.c

編號	題目	整合版頁次	內容定位	程式檔名
S10	有意的空本體與讀到行尾	54	語法片段 / HTML補齊	s10_empty_loop.c
S11	整數常數進位與輸入輸出格式	59、62	語法片段 / HTML補齊	s11_integer_io.c
S12	unsigned 回繞、整數提升與混合比較	60、69	語法片段 / HTML補齊	s12_unsigned_rules.c
S13	先轉型再除法：平均值與零筆資料	70、81、93	語法片段 / HTML補齊	s13_cast_mean.c
S14	float、double、long double 的 I/O 與精度觀察	66–68	語法片段 / HTML補齊	s14_floating_io.c
S15	字元、跳脫字元與 toupper	74、76、77	語法片段 / HTML補齊	s15_char_escape.c
S16	scanf 後殘留的換行與下一個命令	75	語法片段 / HTML補齊	s16_input_buffer.c
S17	typedef、sizeof 與精確寬度型別	80	語法片段 / HTML補齊	s17_typedef_sizeof.c
S18	安全的 unsigned 倒數	93	語法片段 / HTML補齊	s18_reverse_unsigned.c
S19	修正空本體與閉區間接受條件	93	語法片段 / HTML補齊	s19_repair_basics.c
S20	單行訊息長度：原 length.c 的完整化	74、76、77	語法片段 / HTML補齊	s20_message_length.c

S08：原 Chapter 6 Loops.pptx 第45–46頁，奇數累加平方表。S20：原 Chapter 7 Basic Types.pptx 第71–73頁，訊息長度。各題另列其完整化時新增的輸入限制或 EOF 處理。

原17份程式的內容比對

已將原資源包 examples/ 內17個檔案，與本次提供的C01–C16所有程式逐位元組比對，結果全部一致。詳細 SHA-256 與結果列在 ZIP 的 `source_integrity.json`。

閱讀使用說明 →